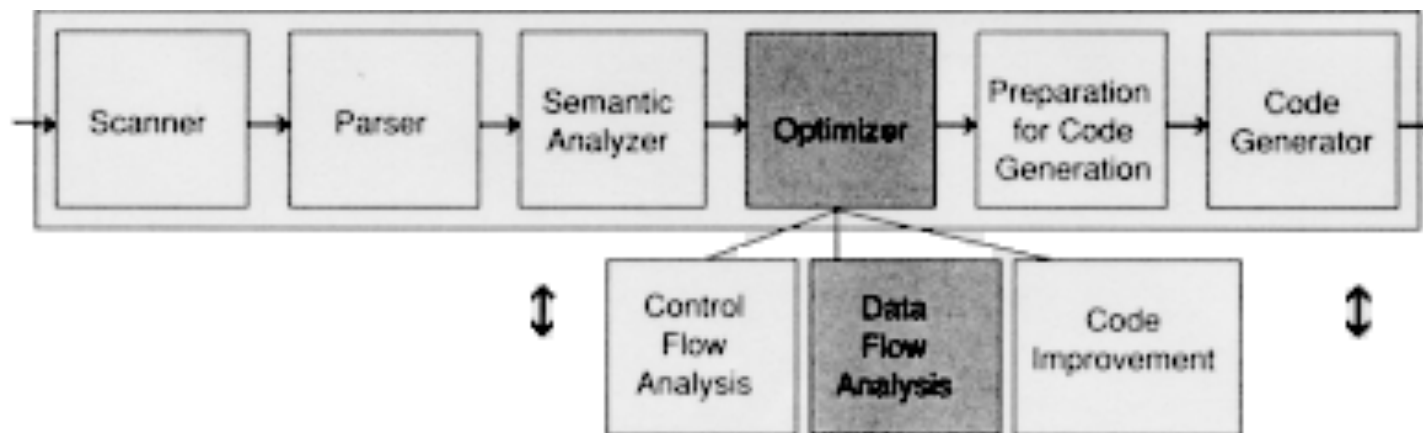


# Dataflow Analyses

# Code Optimization in Compilers

---



## Correctness Above All!

If may seem obvious, but it bears repeating that optimization should not change the correctness of the generated code. Transforming the code to something that runs faster but incorrectly is of little value. It is expected that the unoptimized and optimized variants give the same output for all inputs. This may not hold for an incorrectly written program (e.g., one that uses an uninitialized variable).

# What is Dataflow Analysis: general idea

---

A static analysis technique to track how information flows through a program

- Operates on a graph representation:
  - Typically the Control Flow Graph (CFG)
- Compute program properties (e.g., live variables, constant propagation, reaching definitions)
- Associates information to:
  - nodes (program points) or
  - edges
- Have two main directions:
  - Forward analysis (information flows along execution)
  - Backward analysis (information flows against execution)

Different analyses share the same underlying computational framework

# Common Structure of Dataflow Analyses

---

## Flow equations

- For each node:
  - $IN[n]$  = merge of predecessor (or successor) information
  - $OUT[n]$  = transfer function applied to  $IN[n]$

## Transfer function

- Models the effect of each statement
- Often expressed via gen/kill

## Merge operator

- Combines information from multiple paths
- Common choices:
  - $\cup$  (union)  $\rightarrow$  may analyses
  - $\cap$  (intersection)  $\rightarrow$  must analyses

## Computation

- Iterative solution until a fixed point

# May vs Must Analyses

---

- Two types of dataflow analyses:
  - May (**possible**) analysis
    - Property may hold on some execution paths
    - Examples: reaching definitions, live variables
  - Must (**definitive**) analysis
    - Property holds on all execution paths
    - Examples: available expressions, definite assignment

# Control flow graph

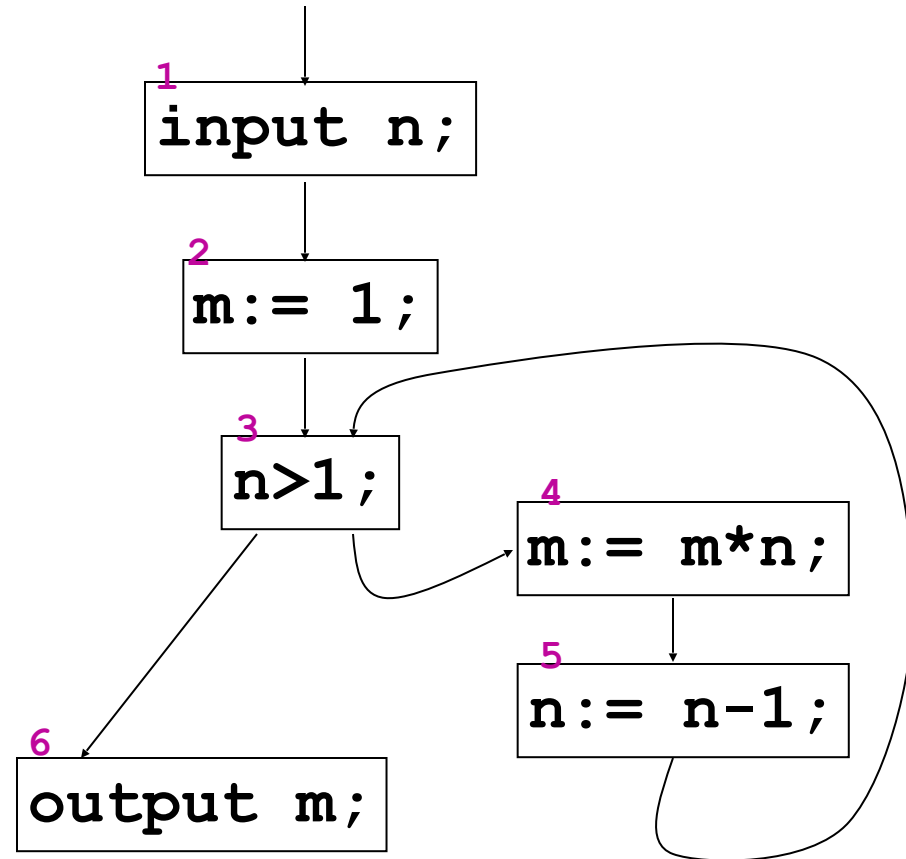
---

- Program commands are encoded by nodes in a control flow graph
- If a command S **may be directly followed** by a command T then the control flow graph must include a direct arc from the node encoding S to the node encoding T

# Example

---

```
[ input n; ]1  
[ m := 1; ]2  
  [ while n>1 do ]3  
    [ m := m * n; ]4  
    [ n := n - 1; ]5  
  [ output m; ]6
```



# Data-Flow analyses

---

We will see several data-flow analyses:

- **Liveness** analysis
- **Reaching definitions** analysis
- **Available expressions** analysis
- **Very busy expressions** analysis

all of them will differ either for **direction flow** or for being **definitive or possible**



# Liveness Analysis

# Liveness or Live Variables Analysis

- We need to translate the source program in the intermediate representation IR that uses a **large** (potentially unbounded) **number of registers**.
- but the program will be executed by a processor with a (finite and) **small number of registers**
- Two variables **a** and **b** can be stored in the same register when it turns out that **a** and **b** are **never simultaneously “used”**

## IR: Three Address Code

---

Three-address instruction has at most three operands and is typically a combination of an assignment and a binary operator.

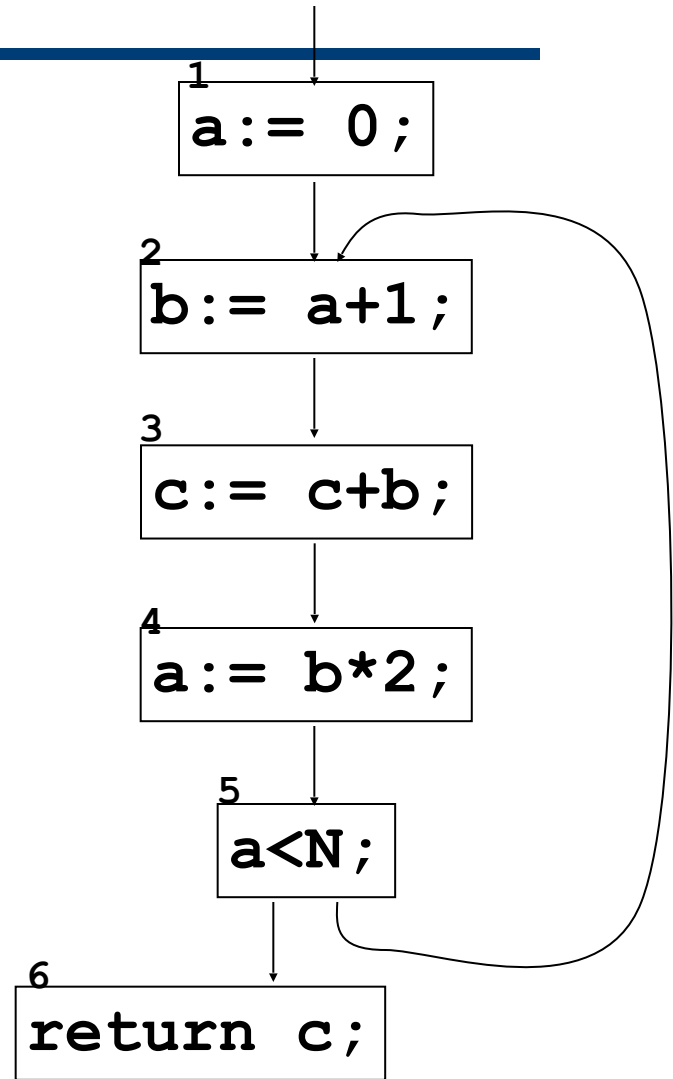
For example:  $t1 := t2 + t3$ .

The name derives from the use of three operands in these statements even though instructions with fewer operands may occur.

# Example

```
a = 0;  
do {  
    b = a+1;  
    c += b;  
    a = b*2;  
}  
while (a<N);  
return c;
```

We want to know if **a** and **b** are simultaneously used.



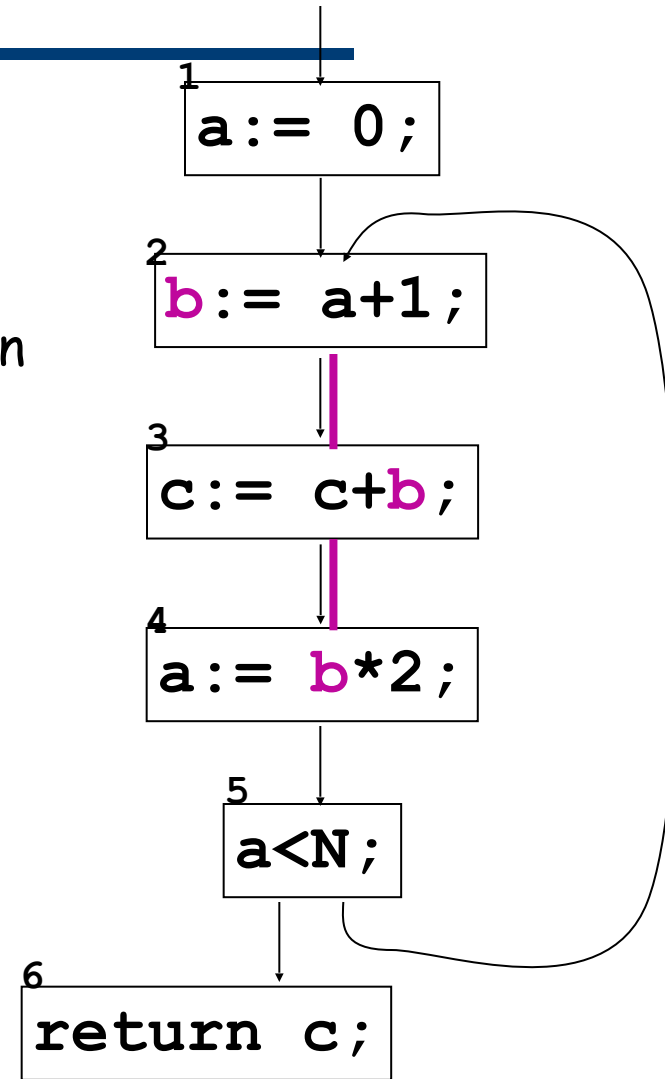
# Live Variables Analysis

---

- A compiler needs to analyze programs in IR in order to find out which variables are simultaneously used
- A variable  $X$  is **live** at the exit of a command  $C$  if  $X$  stores a value which will be **actually used** in the future, that is,  $X$  will be used as R-value with no previous use as L-value
- A variable  $X$  which is not live at the exit of  $C$  is also called **dead** (this information can be used for dead code elimination)
- This is an undecidable property

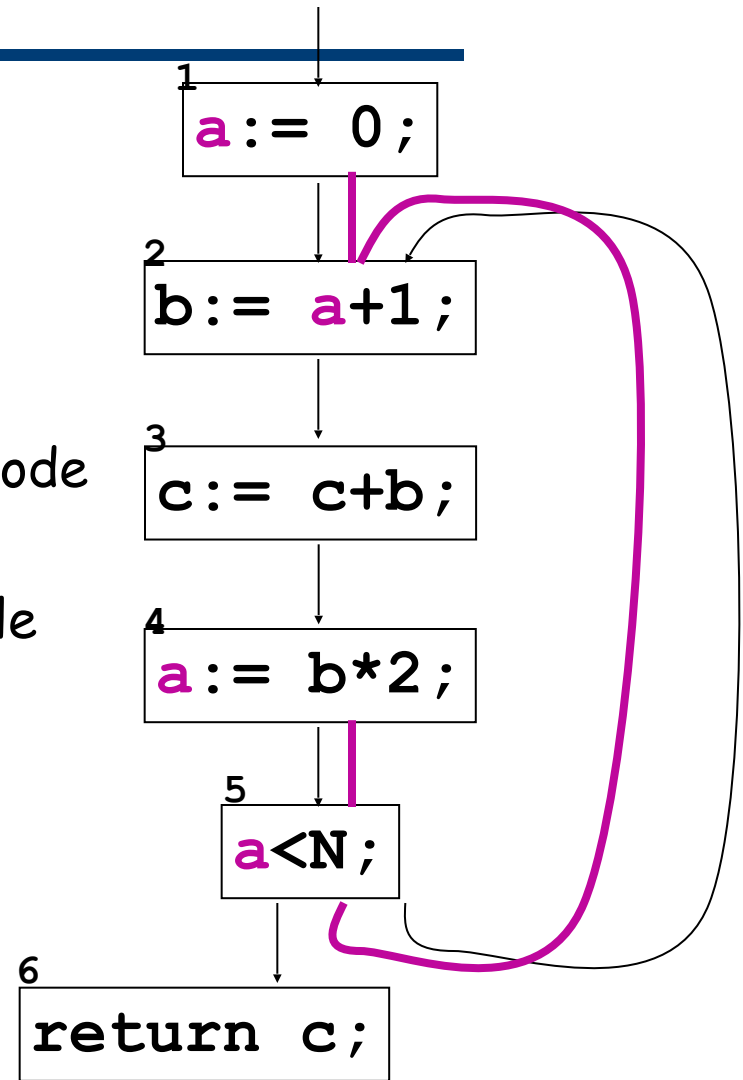
# Back to the example

- A variable  $X$  is live when it stores a value which will be later used with no prior assignment to  $X$
- The “last” use of the variable  $b$  as r-value is in command 4
- The variable  $b$  is used in command 4: it is therefore live along the arc  $3 \rightarrow 4$
- Command 3 does not assign  $b$ , hence  $b$  is live along  $2 \rightarrow 3$
- Command 2 assigns  $b$ . This means that the value of  $b$  along  $1 \rightarrow 2$  will not be used later
- Thus, the “live range” of  $b$  turns out to be:  $\{2 \rightarrow 3, 3 \rightarrow 4\}$



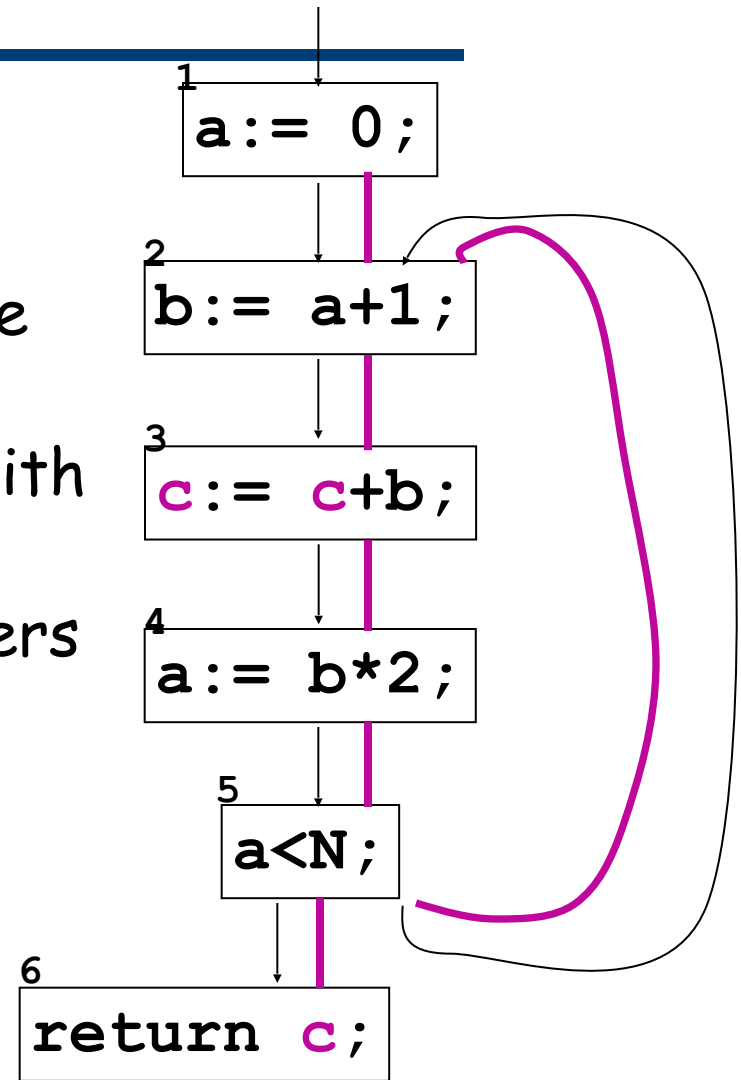
# Live variables

- `a` is live along  $4 \rightarrow 5$  and  $5 \rightarrow 2$
- `a` is live along  $1 \rightarrow 2$
- `a` is not live along  $2 \rightarrow 3$  and  $3 \rightarrow 4$
- Even if the variable `a` stores a value in node 3, this value will not be later used, since node 4 assigns a new value to the variable `a`.

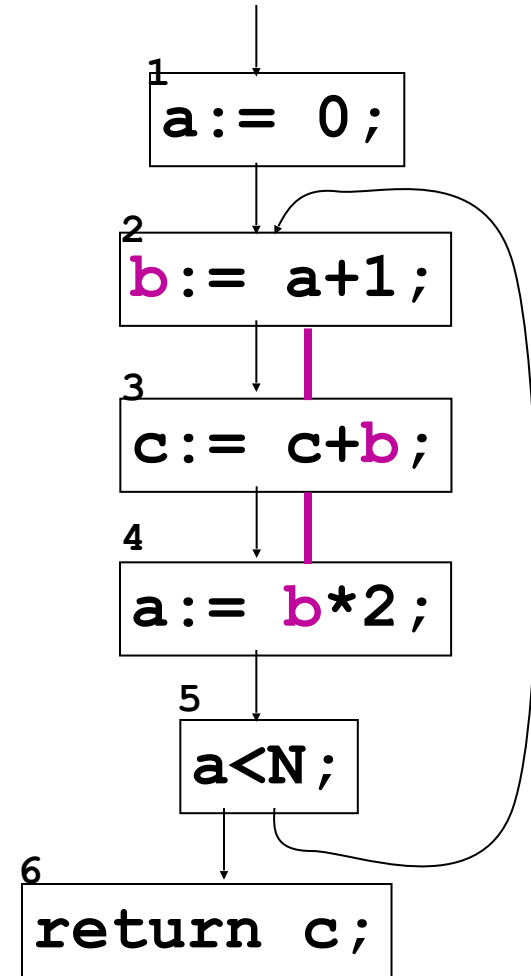
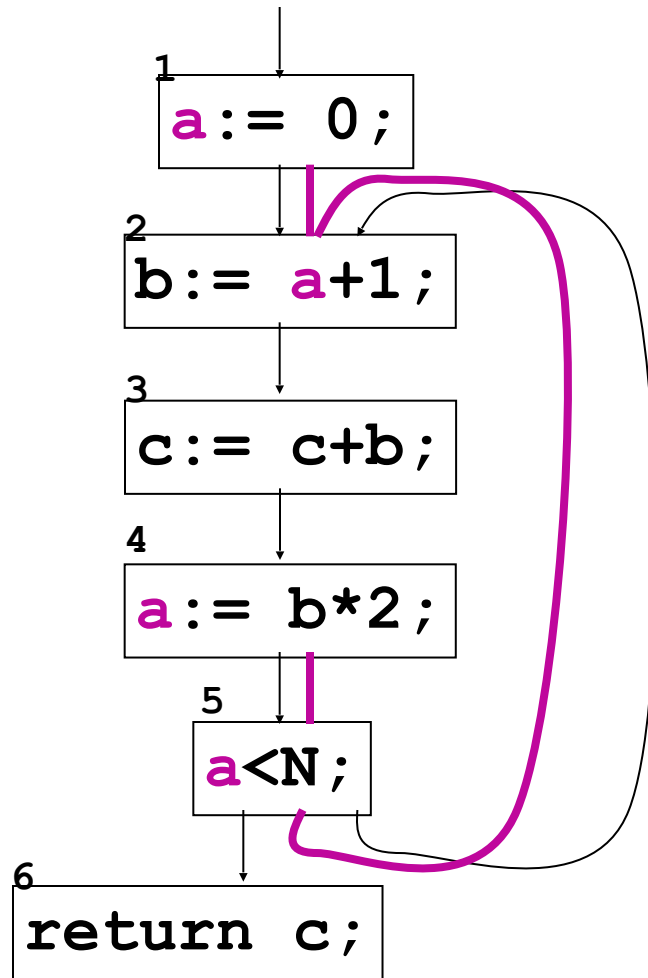


## More on live variables

- **c** is live along all the arcs
- By the way: liveness analysis can be exploited to deduce that if **c** is a local variable then **c** will be used with no prior initialization (this information can be used by compilers to raise a warning message)

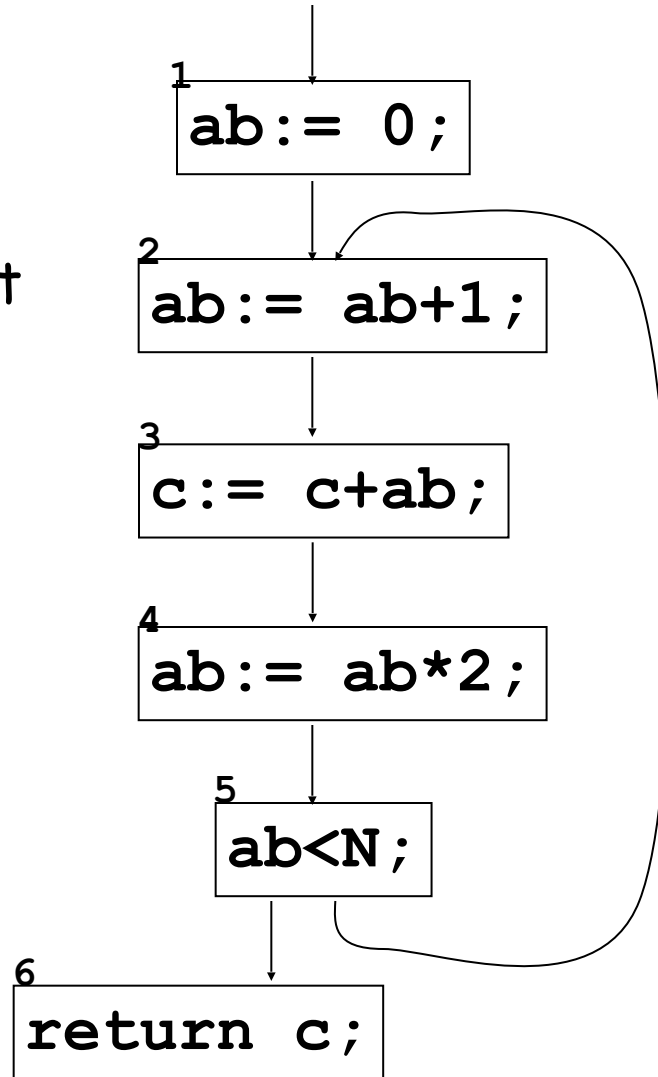






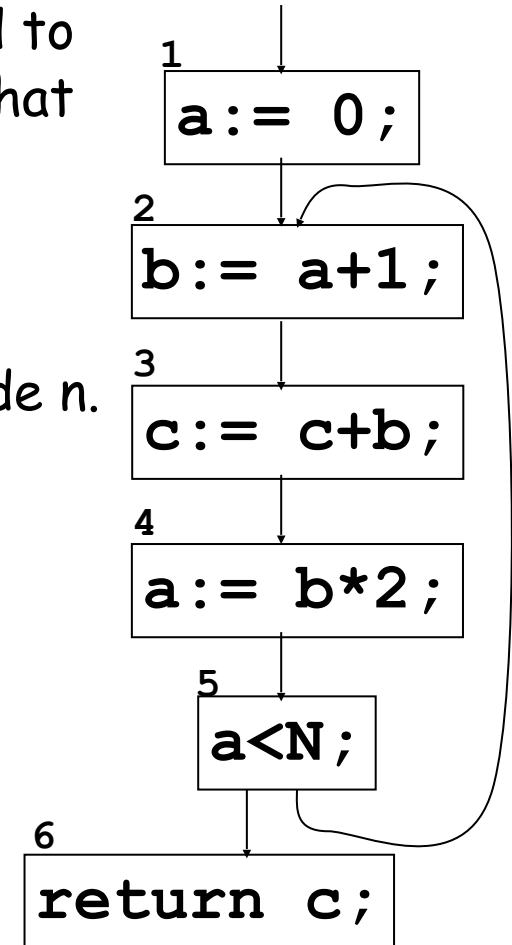
→ Two registers are enough: variables `a` and `b` will be never simultaneously live along the same arc

Variables **a** and **b** will be never simultaneously live along the same arc. Hence, instead of using two distinct variables **a** and **b** we can correctly employ a single variable **ab**



# We need a way to compute live variables

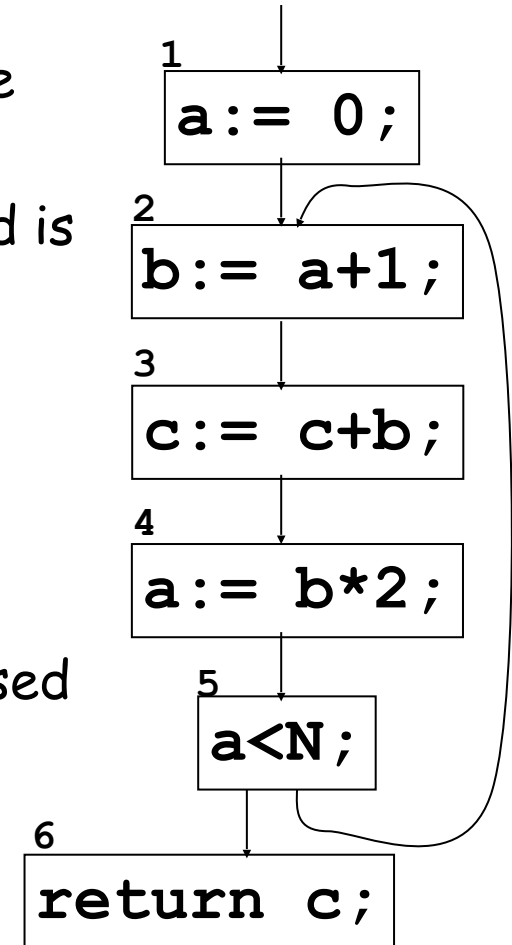
- A CFG has outgoing edges (**out-edges**) that lead to successor nodes, and ingoing edges (**in-edges**) that originate from predecessor nodes.
- **pre[n]** and **post[n]** denote, respectively, the predecessors and successors nodes of some node n.
- As an example, in this CFG:
  - 2 and 6 are successors of node 5 because  
 $5 \rightarrow 6$  and  $5 \rightarrow 2$  are the out-edges of 5
  - 1 and 5 predecessor 2 since  
 $5 \rightarrow 2$  and  $1 \rightarrow 2$  are the in-edges of 2
  - $\text{pre}[2]=\{1,5\}$ ;  $\text{post}[5]=\{2,6\}$ .



# Notation

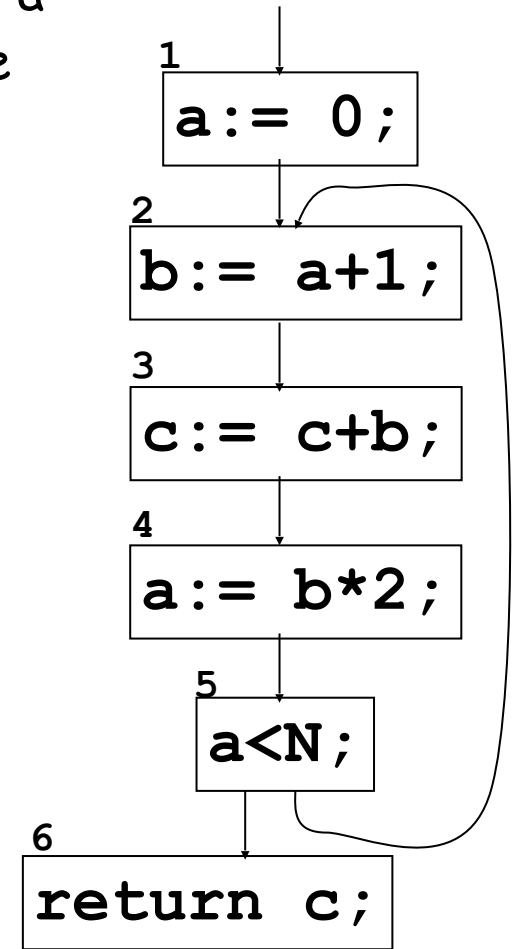
---

- An assignment to some variable (a use of the variable as L-value) is called a **definition** of the variable
- A use of some variable as R-value in a command is called a **use** of this variable
- **def[n]** denotes the set of variables that are defined in the node n
- **use[n]** denotes the set of variables that are used in the node n
- As an example, in this CFG:
  - $\text{def}[3] = \{c\}$ ,  $\text{def}[5] = \emptyset$
  - $\text{use}[3] = \{b, c\}$ ,  $\text{use}[5] = \{a\}$



## Formalization of the property:

- A variable  $x$  is **live** along an arc  $e \rightarrow f$  if there exists a **real execution path**  $P$  from the node  $e$  to some node  $n$  such that:
  - $e \rightarrow f$  is the first arc of such path  $P$
  - $x \in \text{use}[n]$
  - for any node  $n' \neq e$  and  $n' \neq n$  in the path  $P$ ,  
 $x \notin \text{def}[n']$
- A variable  $x$  is **live-out** in some node  $n$  if  $x$  is live along **some** (i.e., at least one) out-edge of  $n$
- A variable  $x$  is **live-in** in some node  $n$  if  $x$  is live along **any** in-edge of  $n$



# Example

---

As an example, in this CFG:

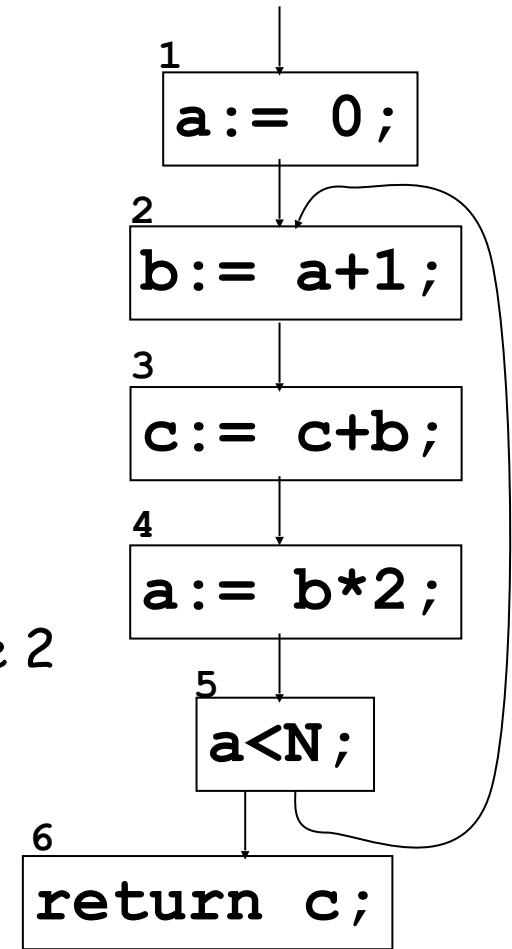
**a** is live along  $1 \rightarrow 2$ ,  $4 \rightarrow 5$  and  $5 \rightarrow 2$

**b** is live along  $2 \rightarrow 3$ ,  $3 \rightarrow 4$

**c** is live along any arc

**a** is live-in in node 2, while it is not live-out in node 2

**a** is live-out in node 5



## Computing an approximation of Liveness property

Let us define the following notation:

$in[n]$  is the set of variables that the static analysis determines to be live-in at node  $n$

$out[n]$  is the set of variables that the static analysis determines to be live-out at node  $n$

## Computing an approximation of Liveness property

Liveness information: the sets  $in[n]$  and  $out[n]$  is computed as an over-approximation in the following way

$n$  node of the CFG

1. If a variable  $x \in use[n]$  then  $x$  is live-in in node  $n$ .  
In other terms, if a node  $n$  uses a variable  $x$  as R-value then this variable  $x$  is live along each arc that enters into  $n$ .

$y := x + z + 2;$   $n$

$$in[n] \supseteq use[n]$$

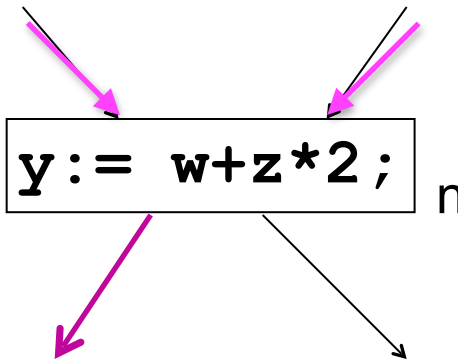


## Computing Liveness

---

2. If a variable  $x$  is live-out in a node  $n$  and  $x \notin \text{def}[n]$  then the variable  $x$  is also live-in in this node  $n$ .

If a variable  $x$  is live for some arc that leaves a node  $n$  and  $x$  is not assigned in  $n$  then  $x$  is live for all the arcs that enter in  $n$



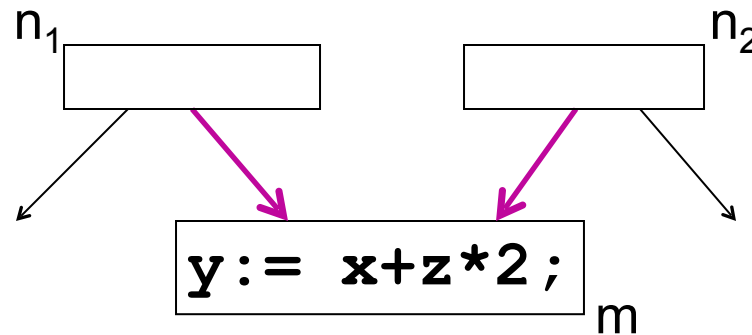
$$\text{in}[n] \supseteq \text{out}[n] - \text{def}[n]$$

## Computing Liveness

---

3. If a variable  $x$  is live-in in a node  $m$  then  $x$  is live-out for all the nodes  $n$  such that  $m \in \text{post}[n]$ .

This is clearly correct by definition.



$$\text{out}[n_1] \supseteq \{ \text{in}[m] \mid m \in \text{post}[n_1] \}$$

$$\text{out}[n_2] \supseteq \{ \text{in}[m] \mid m \in \text{post}[n_2] \}$$

# Dataflow Equations

---

The previous three rules of liveness analysis can be thus formalized by two equations for each node  $n$ :

1.  $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$  (rules 1 and 2)

2.  $\text{out}[n] = \bigcup \{\text{in}[m] \mid m \in \text{post}[n]\}$  (rule 3)

## Correctness of the analysis of Liveness

---

This definition of liveness analysis  $\text{in}[n]$  and  $\text{out}[n]$  is **correct**:

If  $x$  is concretely live-in (live-out) in some node  $n$  then the static analysis will detect that  $x \in \text{in}[n]$  ( $x \in \text{out}[n]$ ):

$$\begin{aligned}\text{in}[n] &\supseteq \text{live-in}[n] \\ \text{out}[n] &\supseteq \text{live-out}[n]\end{aligned}$$

In other terms, no actually live variable is neglected by liveness analysis.

# Correctness in Dragon Book

---

## Why the Available-Expressions Algorithm Works

We need to explain why starting all OUT's except that for the entry block with  $U$ , the set of all expressions, leads to a conservative solution to the data-flow equations; that is, all expressions found to be available really *are* available. First, because intersection is the meet operation in this data-flow schema, any reason that an expression  $x + y$  is found not to be available at a point will propagate forward in the flow graph, along all possible paths, until  $x + y$  is recomputed and becomes available again. Second, there are only two reasons  $x + y$  could be unavailable:

1.  $x + y$  is killed in block  $B$  because  $x$  or  $y$  is defined without a subsequent computation of  $x + y$ . In this case, the first time we apply the transfer function  $f_B$ ,  $x + y$  will be removed from  $\text{OUT}[B]$ .
2.  $x + y$  is never computed along some path. Since  $x + y$  is never in  $\text{OUT}[\text{ENTRY}]$ , and it is never generated along the path in question, we can show by induction on the length of the path that  $x + y$  is eventually removed from IN's and OUT's along that path.

Thus, after changes subside, the solution provided by the iterative algorithm of Fig. 9.20 will include only truly available expressions.

# Computing Liveness

---

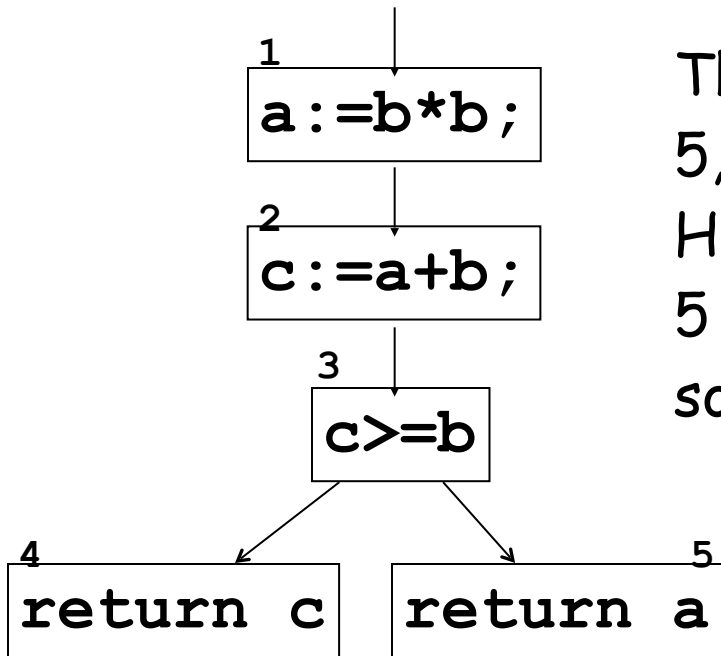
Liveness analysis is **approximate**:

it assumes that each path of the CFG is a **feasible path**  
while this hypothesis is obviously not true

## Computing Liveness

---

Liveness analysis is **approximate**: it assumes that each path of the CFG actually is a **feasible path** while this hypothesis is obviously not true.



The analysis determines that  $a$  is live-in in 5, and therefore  $a$  is live-out in 3.

However, no real execution path from 3 to 5 exists (because  $b + b * b < b$  is always false) so that  $a$  is not really live when exiting 3!

## How can we compute a solution to 1 and 2?

---

1.  $in[n] = use[n] \cup (out[n] - def[n])$

2.  $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

Correctness tells us that  $in[n] \supseteq live-in[n]$  and  $out[n] \supseteq live-out[n]$

But we need a way to compute Live variable analysis



## How can we compute a solution to 1 and 2?

---

1.  $in[n] = use[n] \cup (out[n] - def[n])$
2.  $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

We need to compute for each node the sets that satisfy the previous equations: fix points

- but how can we be sure that such fix-points exist?

It depends on the **domain** and on the **function**!

What do we need?

---

- to settle a domain:  $\text{CPO}_\perp (D, \sqsubseteq)$
- a function  $f : D \rightarrow D$  continuous

then we can apply Kleene 's Theorem

# On finite sets

If a poset  $(D, \sqsubseteq)$  is finite then it is a **CPO**

If a poset  $(D, \sqsubseteq)$  is finite and  $f : D \rightarrow D$  is monotone  
then  $f : D \rightarrow D$  is continuous

1.  $in[n] = use[n] \cup (out[n] - def[n])$
2.  $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

## Which is our domain?

---

Our objects:

Given a node we need to compute the set **in** and the set **out** (sets of variables)

- Let **Vars** be the finite set of variables that occur in the program P to analyze. All possible subsets:  $\mathcal{P}(\text{Vars})$

For each node we need a set **in** and a set **out**:  $\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars})$

Our CFG has **N** nodes, so our **domain** will be

$(\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N$  : N-tuples of pairs of subsets of **Vars**

The **order** :  $\subseteq^{2N}$

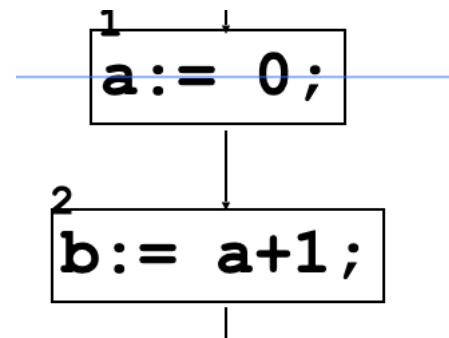
$\langle in_1^1, out_1^1, \dots, in_N^1, out_N^1 \rangle \subseteq^{2N} \langle in_1^2, out_1^2, \dots, in_N^2, out_N^2 \rangle$  iff

$$\forall i, \quad in_i^1 \subseteq in_i^2 \text{ and } out_i^1 \subseteq out_i^2$$

# Example

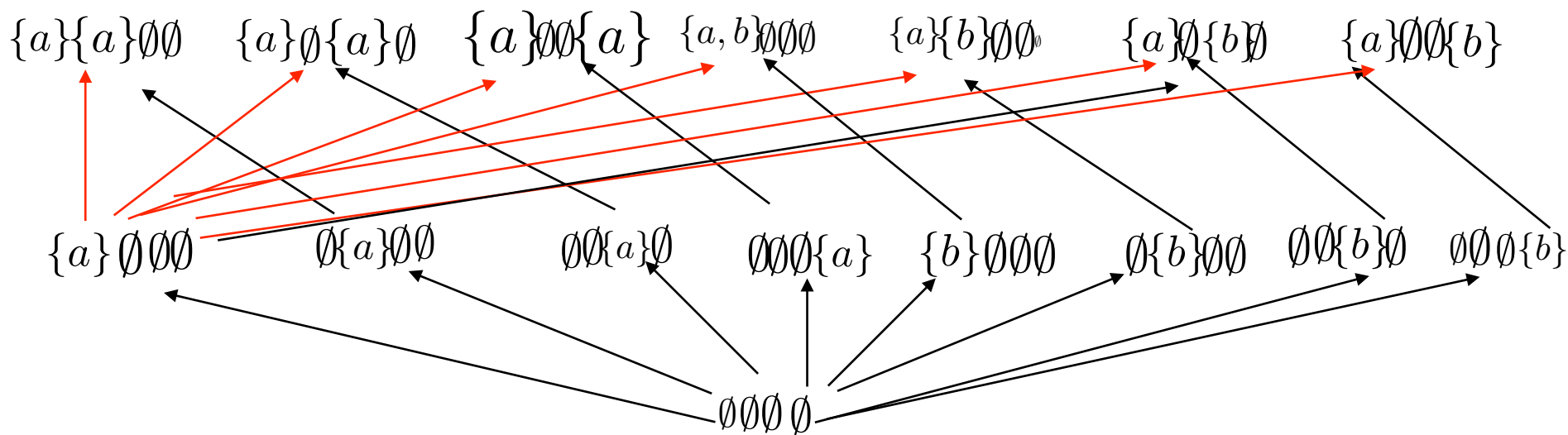
**Vars = {a, b} N=2.**

$\langle (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^2, \subseteq^4 \rangle$  is a finite domain.



$\{a, b\} \{a, b\} \{a, b\} \{a, b\}$

...



## Our domain

---

$$\langle (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^{\mathbb{N}}, \subseteq^{2N} \rangle$$

CPO with bottom?

It is a CPO because it is finite  
bottom?

1.  $in[n] = use[n] \cup (out[n] - def[n])$
2.  $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

## Which is our function?

The map Live:

$(\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N \rightarrow (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N$  defined by

$\text{Live}(\langle in_1, out_1, \dots, in_N, out_N \rangle) =$

$\langle use[1] \cup (out_1 - def[1]), \bigcup_{m \in post[1]} in_m, \dots, use[N] \cup (out_N - def[N]), \bigcup_{m \in post[N]} in_m \rangle$

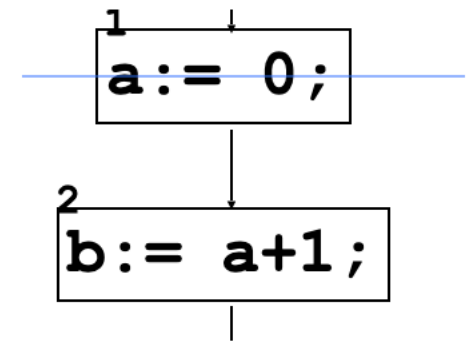
### • Example

$\text{Live}(\langle \{\}\{\}\{\}\{\}\rangle) = \langle \{\}\{\}\{a\}\{\}\rangle$

$\text{Live}(\langle \{\}\{\}\{a\}\{\}\rangle) = \langle \{\}\{a\}\{a\}\{\}\rangle$

$\text{Live}(\langle \{\}\{a\}\{a\}\{\}\rangle) = \langle \{\}\{a\}\{a\}\{\}\rangle$

Note that Live is monotone!



$def(1) = \{a\}, use(1) = \{\}$   
 $def(2) = \{b\}, use(2) = \{a\}$

# Is it continuous?

---

The map Live:

$((\text{Vars}) \times \mathcal{P}(\text{Vars}))^N \rightarrow (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N$  defined by

$\text{Live}(\langle \text{in}_1, \text{out}_1, \dots, \text{in}_N, \text{out}_N \rangle) =$

$\langle \text{use}[1] \cup (\text{out}_1 - \text{def}[1]), \bigcup_{m \in \text{post}[1]} \text{in}_m, \dots, \text{use}[N] \cup (\text{out}_N - \text{def}[N]), \bigcup_{m \in \text{post}[N]} \text{in}_m \rangle$

is continuous?

Yes! because it is **monotone on a finite domain**



## In conclusion

---

The map Live:

$(\mathcal{P}(\mathbf{Vars}) \times \mathcal{P}(\mathbf{Vars}))^N \rightarrow (\mathcal{P}(\mathbf{Vars}) \times \mathcal{P}(\mathbf{Vars}))^N$  defined by

$\text{Live}(\langle \text{in}_1, \text{out}_1, \dots, \text{in}_N, \text{out}_N \rangle) =$

$\langle \text{use}[1] \cup (\text{out}_1 - \text{def}[1]), \bigcup_{m \in \text{post}[1]} \text{in}_m, \dots, \text{use}[N] \cup (\text{out}_N - \text{def}[N]), \bigcup_{m \in \text{post}[N]} \text{in}_m \rangle$

is a monotonic (and therefore continuous) function on the finite CPO

$\langle (\mathcal{P}(\mathbf{Vars}) \times \mathcal{P}(\mathbf{Vars}))^N, \subseteq^{2N} \rangle$  and therefore Live has

a **least fixpoint**

## Why a **least** fixpoint?

---

- Live is a **possible** analysis,

$$\text{in}[n] \supseteq \text{live-in}[n] \text{ and } \text{out}[n] \supseteq \text{live-out}[n]$$

i.e., if a variable  $x$  will be really live in a node  $n$  during some program execution then  $x$  belongs to  $\text{in}[n]$  of all the fixpoints of the function Live

All fixpoints of the equation system is an over-approximation of really live variables.

We want the least fixpoint (the more precise over approximations)

# Conservative Approximation

- How to interpret the output of this static analysis?
- Correctness tells us that:

$$\text{in}[n] \supseteq \text{live-in}[n] \text{ and } \text{out}[n] \supseteq \text{live-out}[n]$$

If the variable  $x$  will be really live in some node  $n$  during some program execution then  $x$  belongs to  $\text{in}[n]$  of all the fixpoints of the function Live (least fixpoint)

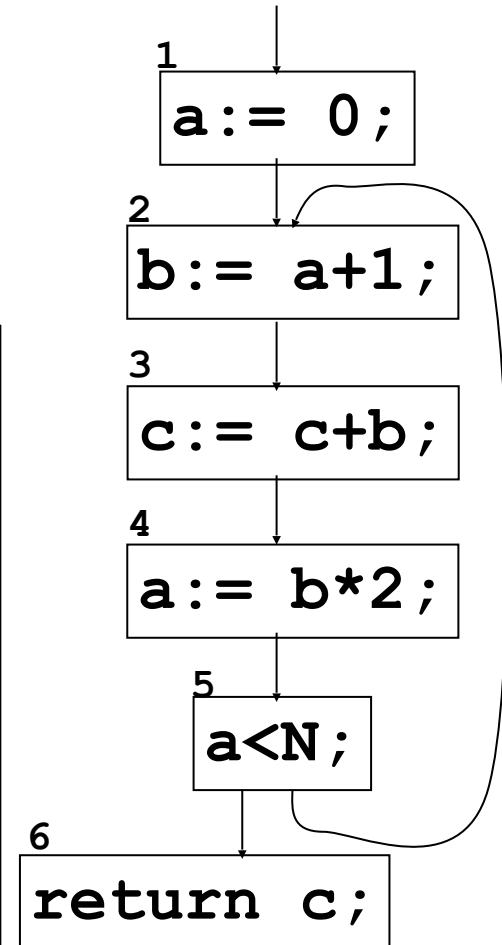
- The converse does not hold: the analysis can tell us that  $x$  is in the computed set  $\text{out}[n]$ , but this does not imply that  $x$  will be necessarily live in  $n$  during some program execution
- In liveness analysis “conservative approximation” means that the analysis may erroneously derive that a variable is live, while the analysis is not allowed to erroneously derive that a variable is “dead” (i.e., not live).
  - ★if  $x \in \text{in}[n]$  then  $x$  **could be live** at program point  $n$ .
  - ★if  $x \notin \text{in}[n]$  then  $x$  is **definitely** dead at program point  $n$ .

```

for all n
  in[n] := {} out[n] := {};
repeat
  for all n (1 to 6)
    in'[n] := in[n]; out'[n] := out[n];
    in[n] := use[n] U (out[n] - def[n]);
    out[n] := U { in[m] | m ∈ post[n] };
until (for all n: in'[n] = in[n] && out'[n] = out[n])

```

|   |     |     | Live <sup>1</sup> |     | Live <sup>2</sup> |     | Live <sup>3</sup> |     |
|---|-----|-----|-------------------|-----|-------------------|-----|-------------------|-----|
|   | use | def | in                | out | in                | out | in                | out |
| 1 |     | a   |                   |     |                   | a   |                   | a   |
| 2 | a   | b   | a                 |     | a                 | b c | a c               | b c |
| 3 | b c | c   | b c               |     | b c               | b   | b c               | b   |
| 4 | b   | a   | b                 |     | b                 | a   | b                 | a   |
| 5 | a   |     | a                 | a   | a                 | a c | a c               | a c |
| 6 | c   |     | c                 |     | c                 |     | c                 |     |

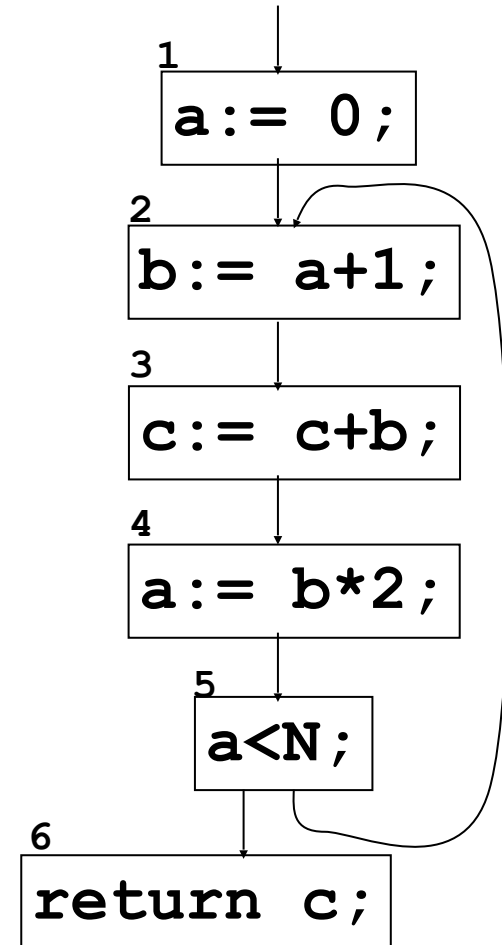


```

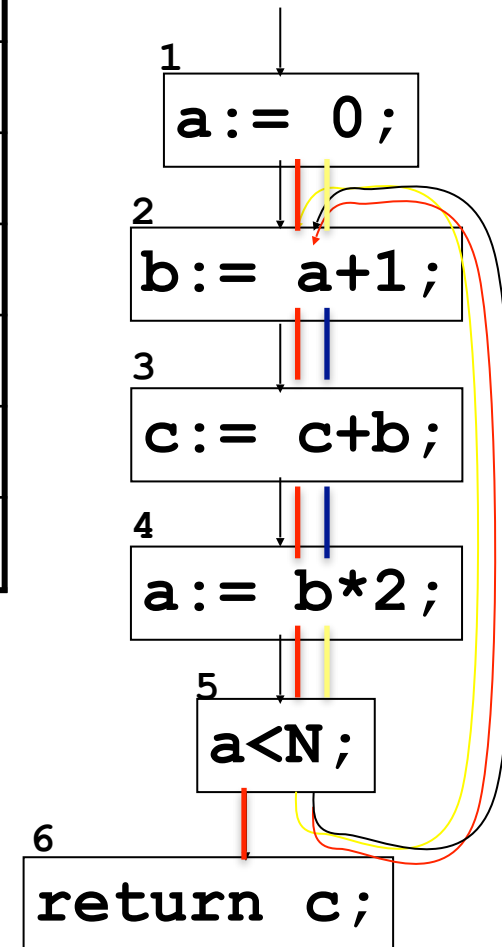
for all n
  in[n]:=?; out[n]:=?;
repeat
  for all n (1 to 6)
    in'[n]:=in[n]; out'[n]:=out[n];
    in[n]:= use[n] U (out[n] - def[n]);
    out[n]:= U { in[m] | m ∈ post[n] };
until (for all n: in'[n]=in[n] && out'[n]=out[n])

```

|   |            | Live <sup>3</sup> | Live <sup>4</sup> | Live <sup>5</sup> |
|---|------------|-------------------|-------------------|-------------------|
|   | use    def | in    out         | in    out         | in    out         |
| 1 | a          | a                 | a c               | c    a c          |
| 2 | a    b     | a c    b c        | a c    b c        | a c    b c        |
| 3 | b c    c   | b c    b          | b c    b          | b c    b          |
| 4 | b    a     | b    a            | b    a c          | b c    a c        |
| 5 | a          | a c    a c        | a c    a c        | a c    a c        |
| 6 | c          | c                 | c                 | c                 |



|   |            | Live <sup>5</sup> |                   | Live <sup>6</sup> |            | Live <sup>7</sup> |            |
|---|------------|-------------------|-------------------|-------------------|------------|-------------------|------------|
|   | use    def | in    out         | in    out         | in    out         | in    out  | in    out         | in    out  |
| 1 | a          | <b>c</b> a c      | c    a c          | c    a c          | c    a c   | c    a c          | c    a c   |
| 2 | a      b   | a c    b c        | a c    b c        | a c    b c        | a c    b c | a c    b c        | a c    b c |
| 3 | b c    c   | b c    b          | b c    b <b>c</b> | b c    b c        | b c    b c | b c    b c        | b c    b c |
| 4 | b      a   | b <b>c</b> a c    | b c    a c        | b c    a c        | b c    a c | b c    a c        | b c    a c |
| 5 | a          | a c    a c        | a c    a c        | a c    a c        | a c    a c | a c    a c        | a c    a c |
| 6 | c          | c                 | c                 | c                 | c          | c                 | c          |



The algorithm thus gives the following output:

out[1]={a,c}, out[2]={b,c}, out[3]={b,c}, out[4]={a,c},  
out[5]={a,c}

In this case, the output of the analysis is precise

# Improvement

---

In this iterative computation, observe that we have to wait for the next iteration in order to exploit the new information computed for in and out on the nodes.

By a suitable reordering of the nodes and by first computing out[n] and then in[n], we are able to converge to the fixpoint in just 3 iteration steps.

```
for all n
  in[n] := ?; out[n] := ?;
repeat
  for all n (6 to 1)
    in'[n] := in[n]; out'[n] := out[n];
    out[n] :=  $\bigcup \{ in[m] \mid m \in post[n] \}$ ;
    in[n] :=  $use[n] \cup (out[n] - def[n])$ ;
until (for all n: in'[n] = in[n] && out'[n] = out[n])
```

```

for all n
  in[n]:=?; out[n]:=?;
repeat
  for all n (6 to 1)
    in'[n]:=in[n]; out'[n]:=out[n];
    out[n]:= U { in[m] | m ∈ post[n] };
    in[n]:= use[n] U (out[n] - def[n]);
until (for all n: in'[n]=in[n] && out'[n]=out[n])

```

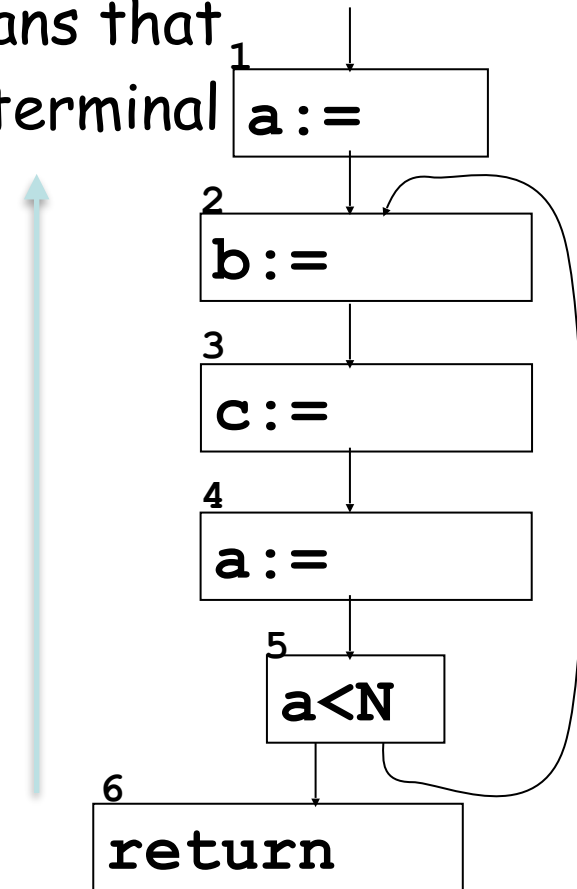
|          |          | Live <sup>1</sup> | Live <sup>2</sup> | Live <sup>3</sup> |
|----------|----------|-------------------|-------------------|-------------------|
|          | use  def | out  in           | out  in           | out  in           |
| <b>6</b> | c        | c                 | c                 | c                 |
| <b>5</b> | a        | c  a c            | a c  a c          | a c  a c          |
| <b>4</b> | b  a     | a c  b c          | a c  b c          | a c  b c          |
| <b>3</b> | b c  c   | b c  b c          | b c  b c          | b c  b c          |
| <b>2</b> | a  b     | b c  a c          | b c  a c          | b c  a c          |
| <b>1</b> | a        | ac  c             | ac  c             | ac  c             |



# Backward Analysis

As shown by the previous example, Live Variable Analysis is a “backward” analysis. This means that information propagates “backward” from terminal nodes to initial nodes:

1.  $\text{in}[n]$  can be computed from  $\text{out}[n]$ ;
2.  $\text{out}[n]$  can be computed from  $\text{in}[m]$  for all the nodes  $m$  that are successors of  $n$ .

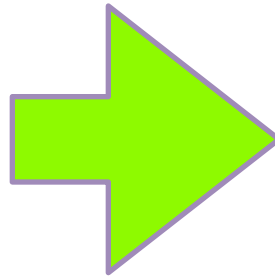


# Application: Dead Code Elimination

---

```
i := 0;
t3 := 0;
while i <= n do
  j := 0;
  t2 := t3;
  while j <= m do
    t1 := t3 + j;
    temp := Base(A) + t1;
    Cont(temp) := Cont(Base(B) + t1)
                + Cont(Base(C) + t1);
    j := j+1
  od;
  i := i+1;
  t3 := t3 + (m+1)
od
```

dead variable



# Reaching Definitions (Reaching Assignment) Analysis

One of the more useful data-flow analysis

```
d1 : y := 3  
d2 : x := y
```

d1 is a reaching definition for d2

```
d1 : y := 3  
d2 : y := 4  
d3 : x := y
```

d1 is no longer a reaching definition for d3, because d2 kills its reach:  
the value defined in d1 is no longer available and cannot reach d3

A definition d at point i reaches a point p if there is a path from the point i to p such that d is not killed (redefined) along that path

## Reaching definitions

---

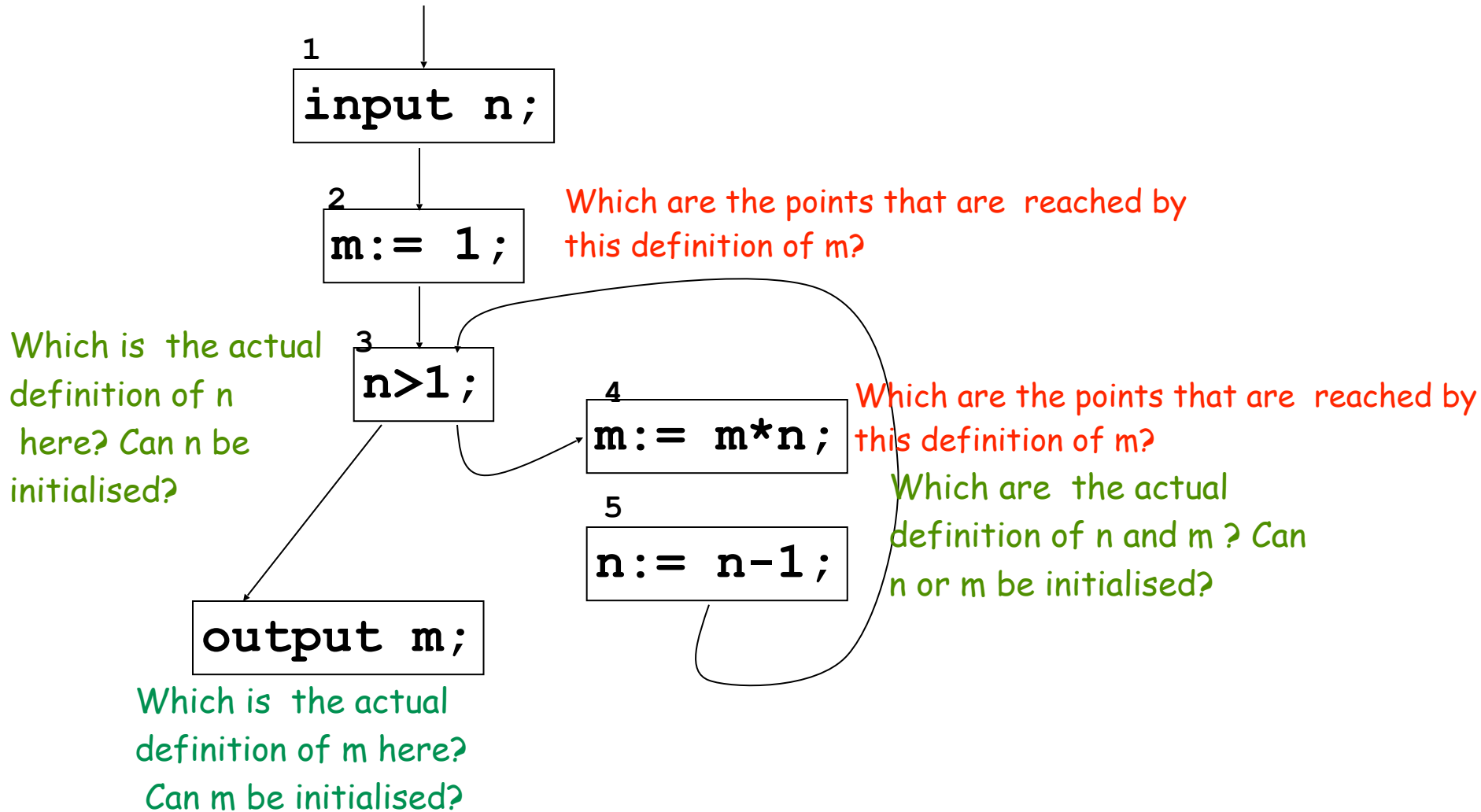
This information is very useful

- The compiler can know whether  $x$  is a constant at point  $p$
- The debugger can tell whether is possible that  $x$  is an undefined variable at point  $p$

## Reaching definitions

- Given a program point  $n$ , which **definitions** are actual - not successively overwritten by a different assignment - when the execution reaches  $n$ ?  
And when the execution leaves  $n$ ?
- A program point may clearly **"generate"** new definitions
- A program point  $n$  may **"kill"** a definition:  
if  $n$  is an assignment  $x := \text{exp}$  then  $n$  kills all the assignments to the variable  $x$  which are actual in input to  $n$
- We are thus interested in computing input and output reaching definitions for any program point

# The intuition: the factorial of n



## Formalization of the reaching definition property

- The property can be represented by sets of pairs:  
 $\{(x,p) \mid x \in \mathbf{Vars}, p \text{ is a program point}\} \in \mathcal{P}(\mathbf{Vars} \times \mathbf{Points})$   
where  $(x,p)$  means that the variable  $x$  is assigned at program point  $p$
- For each program point, this dataflow analysis computes a set of such pairs
- The meaning of a pair  $(x,p)$  in the set for a program point  $q$  is that the assignment of  $x$  at point  $p$  is actual at point  $q$
- $?$  is a special symbol that we add to **Points** and we use to represent the fact that a variable  $x$  is not initialized.
- The set  $\iota = \{(x,?) \mid x \in \mathbf{Vars}\}$  therefore denotes that all the program variables are not initialized.

# The domain for Reaching Definitions Analysis

**Vars** is the (finite) set of variables occurring in the program P.

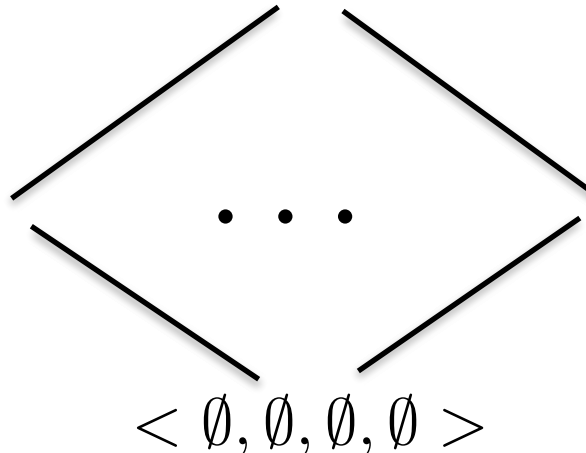
Let **N** be the number of nodes of the CFG of P.

Let **Points**={?,1,...N}.

$$\langle \mathcal{P}(\text{Vars} \times \text{Points}) \times \mathcal{P}(\text{Vars} \times \text{Points}) \rangle^{\text{N}}, \subseteq^{2N} \rangle$$

- Example Vars={a,b} e N=2

$$\langle S = \{(a,?), (a,1), (a,2), (b,?), (b,1), (b,2)\}, S, S, S \rangle$$





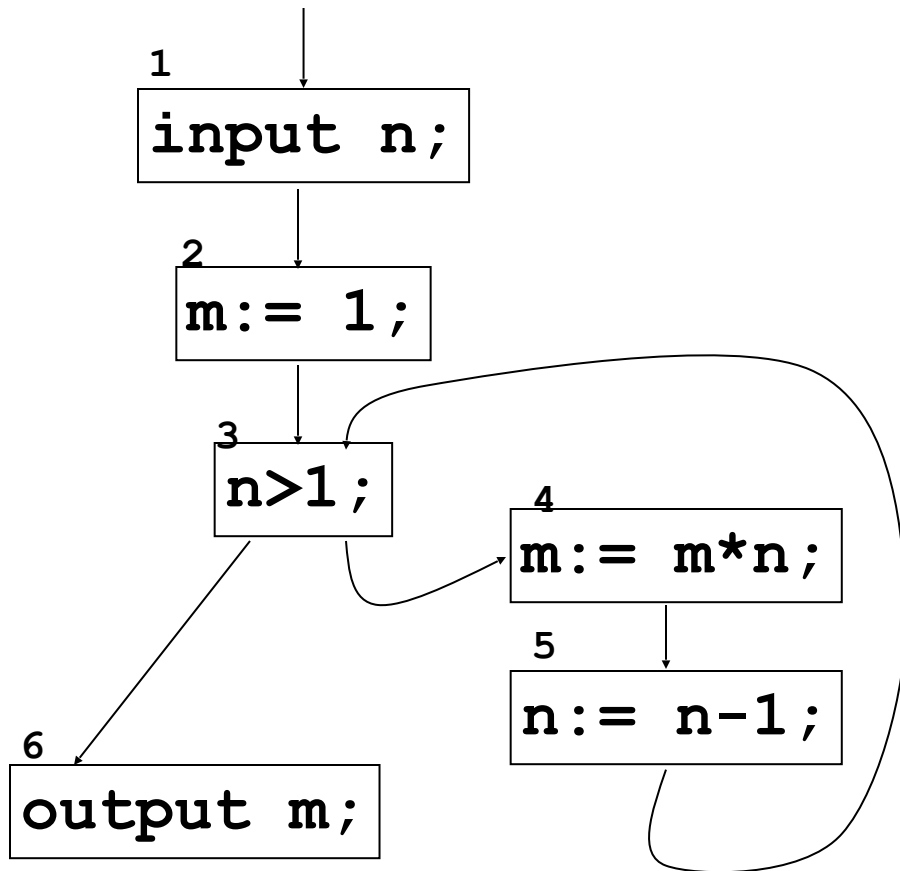
# Specification

---

- $\text{kill}_{\text{RD}}[p] = \begin{cases} \{(x,q) \mid q \in \mathbf{Points} \text{ and } \{x\} = \text{def}[q]\} & \text{if } \{x\} = \text{def}[p] \\ \emptyset & \text{if } \emptyset = \text{def}[p] \end{cases}$
- $\text{gen}_{\text{RD}}[p] = \begin{cases} \{(x,p)\} & \text{if } \{x\} = \text{def}[p] \\ \emptyset & \text{if } \emptyset = \text{def}[p] \end{cases}$

As usual,  $\text{def}[p] = \{x\}$  when the command in the point  $p$  is an assignment  $x := \text{exp}$

# Kill and Gen



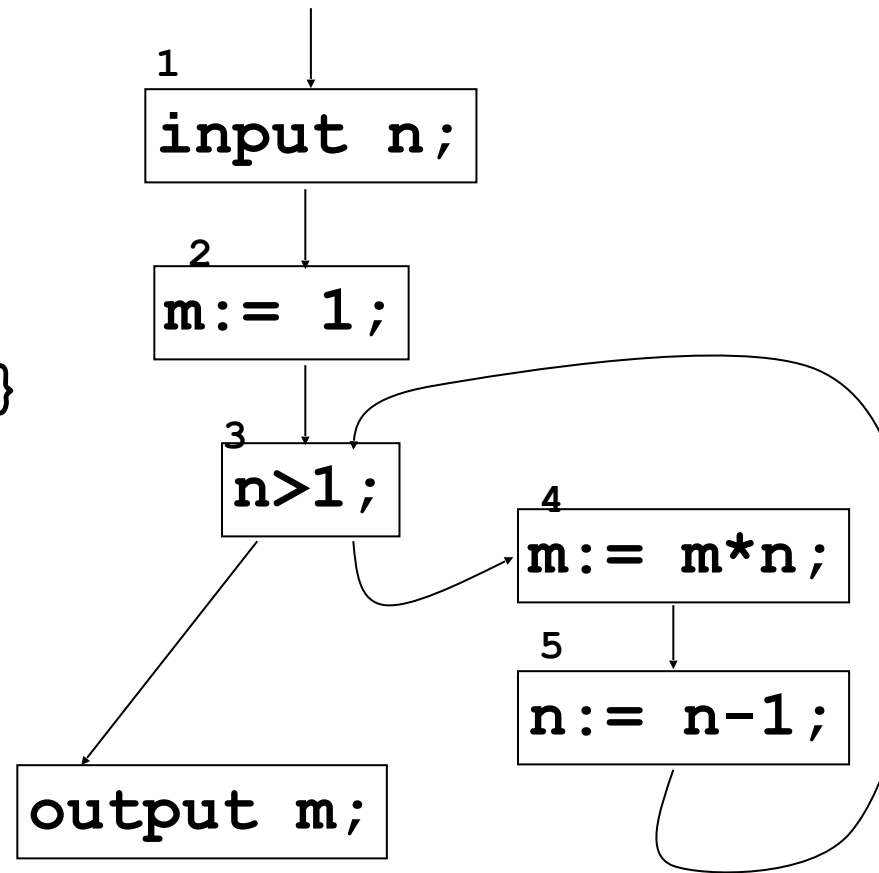
|   | kill <sub>RD</sub>  | gen <sub>RD</sub> |
|---|---------------------|-------------------|
| 1 |                     |                   |
| 2 | (m,?)(m,2)<br>(m,4) | (m,2)             |
| 3 |                     |                   |
| 4 | (m,?)(m,2)<br>(m,4) | (m,4)             |
| 5 | (n,?) (n,5)         | (n,5)             |
| 6 |                     |                   |

# Specification

- Reaching definitions analysis is specified by equations:

$$RD_{\text{entry}}(p) = \begin{cases} \{(x, ?) \mid x \in \text{VARS}\} & \text{if } p \text{ is initial} \\ \bigcup \{RD_{\text{exit}}(q) \mid q \in \text{pre}[p]\} & \text{if } p \text{ is not initial} \end{cases}$$

$$RD_{\text{exit}}(p) = (RD_{\text{entry}}(p) \setminus \text{kill}_{RD}[p]) \cup \text{gen}_{RD}[p]$$



## The solution of the previous system

Once again the solution for the equations in the previous system requires the existence of a fix point

We can apply the Kleene theorem if we have

- a) a continuous function on
- b) a CPO with bottom

## Point b

---

$$\langle (\mathcal{P}(\text{Vars} \times \text{Points}) \times \mathcal{P}(\text{Vars} \times \text{Points}))^{\mathbb{N}}, \subseteq^{2^N} \rangle$$

is a CPO with bottom?

It is a CPO because it is finite  
Bottom?

## Point a: the function

---

The map Reach:

$$\langle \mathcal{P}_{(\text{Vars} \times \text{Points})} \times \mathcal{P}_{(\text{Vars} \times \text{Points})} \rangle^N \rightarrow \langle \mathcal{P}_{(\text{Vars} \times \text{Points})} \times \mathcal{P}_{(\text{Vars} \times \text{Points})} \rangle^N$$

defined by

(assuming 1 is the only initial node)

$$\text{Reach}(\langle \text{RD}_{\text{entry}_1}, \text{RD}_{\text{exit}_1}, \dots, \text{RD}_{\text{entry}_N}, \text{RD}_{\text{exit}_N} \rangle) =$$

$$\langle \{(x, ?) \mid x \in \text{VARS}\}, (\text{RD}_{\text{entry}_1} \setminus \text{kill}_{\text{RD}}[1]) \cup \text{gen}_{\text{RD}}[1],$$

$$\cup \{\text{RD}_{\text{exit}_2} \mid m \in \text{pre}[2]\}, (\text{RD}_{\text{entry}_2} \setminus \text{kill}_{\text{RD}}[2]) \cup \text{gen}_{\text{RD}}[2],$$

....,

$$\cup \{\text{RD}_{\text{exit}_m} \mid m \in \text{pre}[N]\}, (\text{RD}_{\text{entry}_N} \setminus \text{kill}_{\text{RD}}[N]) \cup \text{gen}_{\text{RD}}[N] \rangle$$

# Point a

$$\text{Reach}(\langle \text{RDentry}_1, \text{RDexit}_1, \dots, \text{RDentry}_N, \text{RDexit}_N \rangle =$$

$$\langle \{(x, ?) \mid x \text{ in } \text{VARS}\}, (\text{RD}_{\text{entry1}} \setminus \text{kill}_{\text{RD}}[1]) \cup \text{gen}_{\text{RD}}[1],$$

$$\cup \{\text{RD}_{\text{exit2}} \mid m \text{ in } \text{pre}[2]\}, (\text{RD}_{\text{entry2}} \setminus \text{kill}_{\text{RD}}[2]) \cup \text{gen}_{\text{RD}}[2]$$

$$\dots,$$

$$\cup \{\text{RD}_{\text{exitm}} \mid m \text{ in } \text{pre}[N]\}, (\text{RD}_{\text{entryN}} \setminus \text{kill}_{\text{RD}}[N]) \cup \text{gen}_{\text{RD}}[N] \rangle$$

## • Example

$$\text{Reach}(\langle \{(a, ?)\}, \{\}, \{\}, \{\} \rangle) = \langle \{(a, ?)(b, ?)\}, \{(a, 1)(b, ?)\}, \{(a, 1)(b, ?)\}, \{(a, 1)(b, 2)\} \rangle$$

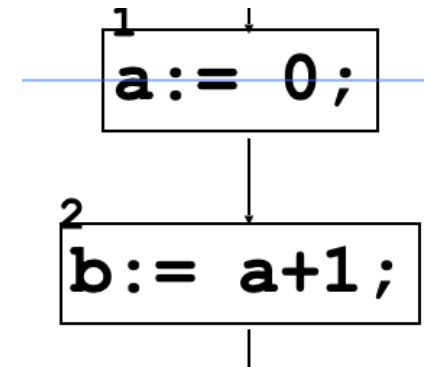
$$\subseteq^{2N}$$

$$\text{Reach}(\langle \{(a, ?)(a, 2)\}, \{(a, 2)\}, \{\}, \{(b, 1)\} \rangle) = \langle \{(a, ?)(b, ?)\}, \{(a, 1)(b, ?)\}, \{(a, 1)(b, ?)\}, \{(a, 1)(b, 2)\} \rangle$$

$$\subseteq^{2N}$$

Note that Reach is monotone!

Since it is monotone on a finite domain then it is continuous



$$\text{kill}_{\text{RD}}(1) = \{(a, ?)\}, \text{gen}_{\text{RD}}(1) = \{(a, 1)\}$$

$$\text{kill}_{\text{RD}}(2) = \{(b, ?)\}, \text{gen}_{\text{RD}}(2) = \{(b, 2)\}$$

## Why a **least** fix point

---

RD analysis is **possible**,

if an assignment  $x:=a$  in some point  $q$  is really actual in entry to some point  $p$  then

$$(x, q) \in \text{RD}_{\text{entry}}(p)$$

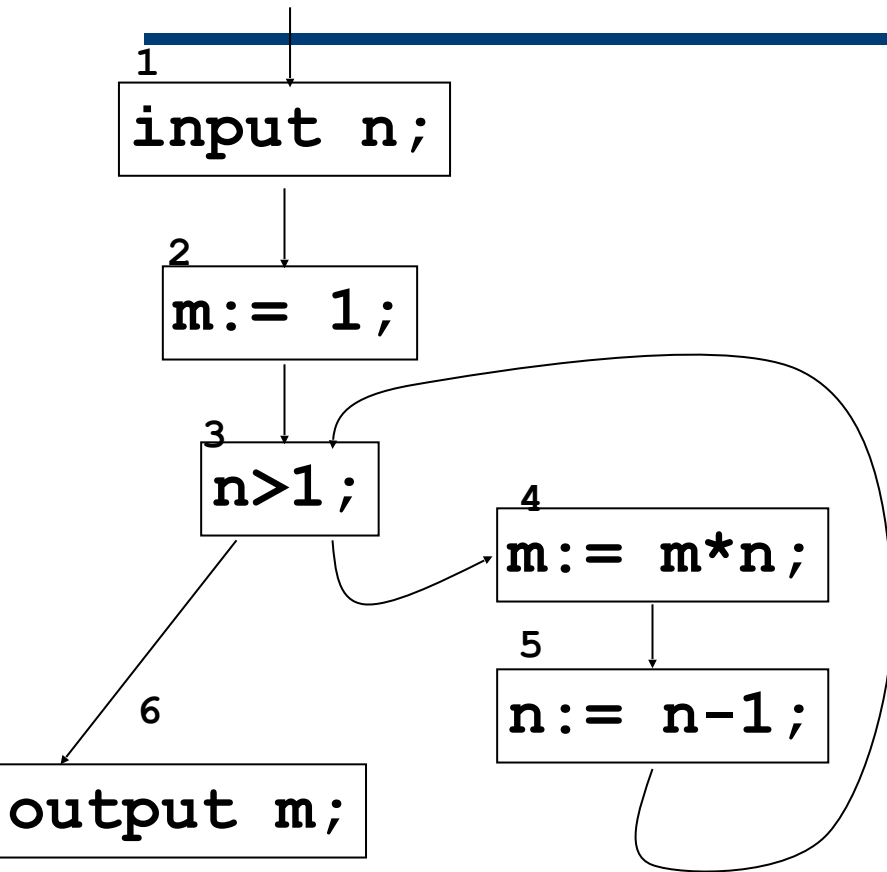
The vice versa does not hold

All fixpoints of the above equation system is an over-approximation of really reaching definitions.

Computing the least fixpoint gives a more precise over approximation



First iteration:

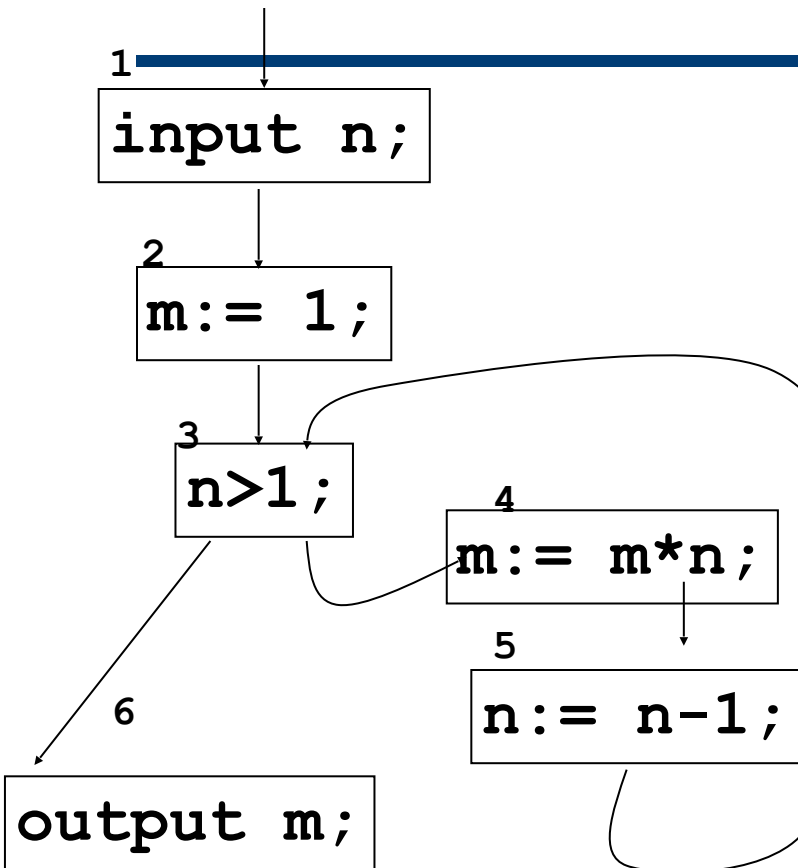


$RD_{\text{entry}}(p) = \{(x, ?) \mid x \text{ in Vars}\}$ , if  $p$  is initial  
 $RD_{\text{entry}}(p) = U\{RD_{\text{exit}}(q) \mid q \text{ in pre}[p]\}$ , otherwise  
 $RD_{\text{exit}}(p) = (RD_{\text{entry}}(p) \setminus \text{kill}_{RD}[p]) \cup \text{gen}_{RD}[p]$

|   | kill                       | gen      |
|---|----------------------------|----------|
| 2 | $(m, ?)(m, 2)$<br>$(m, 4)$ | $(m, 2)$ |
| 4 | $(m, ?)(m, 2)$<br>$(m, 4)$ | $(m, 4)$ |
| 5 | $(n, ?)(n, 5)$             | $(n, 5)$ |

$RD_{\text{entry}}(1) = \{(n, ?), (m, ?)\}$   
 $RD_{\text{exit}}(1) = \{(n, ?), (m, ?)\}$   
 $RD_{\text{entry}}(2) = \{(n, ?), (m, ?)\}$   
 $RD_{\text{exit}}(2) = \{(n, ?), (m, 2)\}$   
 $RD_{\text{entry}}(3) = \{(n, ?), (m, 2)\}$   
 $RD_{\text{exit}}(3) = \{(n, ?), (m, 2)\}$   
 $RD_{\text{entry}}(4) = \{(n, ?), (m, 2)\}$   
 $RD_{\text{exit}}(4) = \{(n, ?), (m, 4)\}$   
 $RD_{\text{entry}}(5) = \{(n, ?), (m, 4)\}$   
 $RD_{\text{exit}}(5) = \{(n, 5), (m, 4)\}$   
 $RD_{\text{entry}}(6) = \{(n, ?), (m, 2)\}$   
 $RD_{\text{exit}}(6) = \{(n, ?), (m, 2)\}$

Second iteration:



|   |                            |          |
|---|----------------------------|----------|
| 2 | $(m, ?)(m, 2)$<br>$(m, 4)$ | $(m, 2)$ |
| 4 | $(m, ?)(m, 2)$<br>$(m, 4)$ | $(m, 4)$ |
| 5 | $(n, ?)(n, 5)$             | $(n, 5)$ |

$$RD_{\text{entry}}(1) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{exit}}(1) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{entry}}(2) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{exit}}(2) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{entry}}(3) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{exit}}(3) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{entry}}(4) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{exit}}(4) = \{(n, ?), (m, 4)\}$$

$$RD_{\text{entry}}(5) = \{(n, ?), (m, 4)\}$$

$$RD_{\text{exit}}(5) = \{(n, 5), (m, 4)\}$$

$$RD_{\text{entry}}(6) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{exit}}(6) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{entry}}(1) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{exit}}(1) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{entry}}(2) = \{(n, ?), (m, ?)\}$$

$$RD_{\text{exit}}(2) = \{(n, ?), (m, 2)\}$$

$$RD_{\text{entry}}(3) = \{(n, ?), (m, 2), (n, 5)(m, 4)\}$$

$$RD_{\text{exit}}(3) = \{(n, ?), (m, 2), (n, 5)(m, 4)\}$$

$$RD_{\text{entry}}(4) = \{(n, ?), (m, 2), (n, 5)(m, 4)\}$$

$$RD_{\text{exit}}(4) = \{(n, ?), (n, 5)(m, 4)\}$$

$$RD_{\text{entry}}(5) = \{(n, ?), (n, 5)(m, 4)\}$$

$$RD_{\text{exit}}(5) = \{(n, 5), (m, 4)\}$$

$$RD_{\text{entry}}(6) = \{(n, ?), (m, 2), (n, 5)(m, 4)\}$$

$$RD_{\text{exit}}(6) = \{(n, ?), (m, 2), (n, 5)(m, 4)\}$$

fix point!

$$RD_{\text{entry}}(p) = \{(x, ?) \mid x \text{ in Vars}\}, \text{ if } p \text{ is initial}$$

$$RD_{\text{entry}}(p) = \bigcup \{RD_{\text{exit}}(q) \mid q \text{ in pre}[p]\}, \text{ otherwise}$$

$$RD_{\text{exit}}(p) = (RD_{\text{entry}}(p) \setminus \text{kill}_{RD}[p]) \cup \text{gen}_{RD}[p]$$

# RD analysis

---

- RD analysis is forward and **possible**,  
i.e., if an assignment  $x:=a$  in some point  $q$  is really actual in entry  
to some point  $p$  then  
 $(x,q) \in \text{RD}_{\text{entry}}(p)$  (while the vice versa does not hold).

How can we use this?

- If the analysis tells us that a variable is undefined (that is we just have the pair  $(x,?)$  ) then it is
- Loop invariant code motions

## Application: Loop invariant code motion

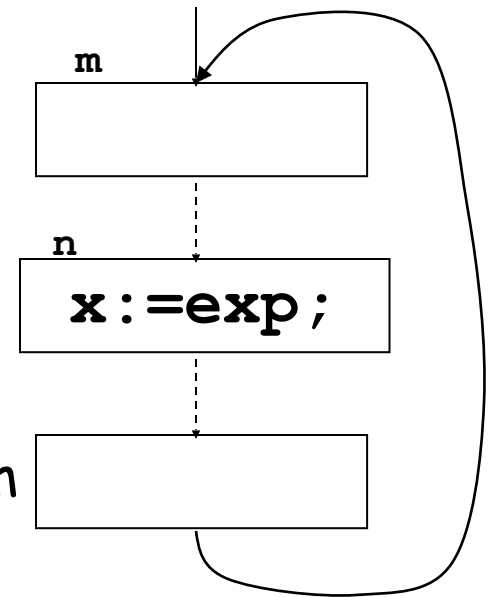
---

Consider a loop where:

1.  $m$  is the entry point
2. an inner point  $n$  contains an assignment  $x := \text{exp}$
3. if for any variable  $y$  occurring in  $\text{exp}$  (i.e.  $y \in \text{vars}(\text{exp})$ ) and for any program point  $p$ , we have that

$$(y, p) \in \text{RD}_{\text{entry}}(m) \iff (y, p) \in \text{RD}_{\text{entry}}(n)$$

then, the assignment  $x := \text{exp}$  can be correctly moved out as preceding the entry point of the loop



## Application: Loop invariant code motion

### Loop-invariant code motion

```
y:=3; z:=5;  
for(int i=0; i<9; i++) {  
    x = y + z;  
    a[i] = 2*i + x;  
}
```

```
y:=3; z:=5;  
x = y + z;  
for(int i=0; i<9; i++) {  
    a[i] = 2*i + x;  
}
```

## Available Expressions Analysis

---

Let  $p$  be a program point. For each execution path ending in  $p$ , we want track the expressions that have already been evaluated and then not modified.

These are called **available expressions**

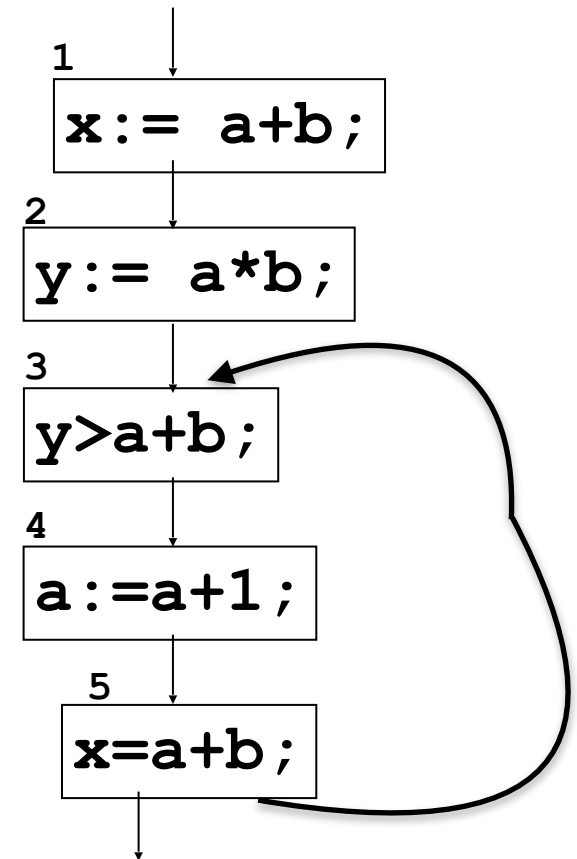
# Example

---

```
x:=a+b;  
y:=a*b;  
while y>a+b  
do (a:=a+1;  
    x:=a+b;)
```

when the execution reaches 3, the expression  $a+b$  is available, since it has been previously evaluated (in point 1 for the first iteration of the while-loop and in point 5 for the next iterations) and does not need to be evaluated again in 3

- This analysis can be therefore used to avoid re-evaluations of available expressions



# The domain

---

Let

$E = \{ e \mid e \text{ is a sub-expressions/expression appearing in program } P \}$

Let  $N$  be the number of nodes of the CFG of  $P$

$\langle (\mathcal{P}(E) \times \mathcal{P}(E))^N, \subseteq^{2N} \rangle$  is a finite domain



# $Kill_{AE}$ and $Gen_{AE}$

---

- An expression  $e$  in the set  $E$  is killed in a program point  $p$  ( $e$  is in  $kill_{AE}(p)$ ) if a variable occurring in  $e$  is modified (i.e., it is defined by an assignment) by the command in  $p$ .

$$kill_{AE}([x:=e']p) = \{e \text{ in } E \mid x \in \text{vars}(e)\}$$

- An expression  $e$  is generated in a program point  $p$  ( $e$  is in  $gen_{AE}(p)$ ) if  $e$  is evaluated in  $p$  and no variable occurring in  $e$  is modified in  $p$ .

$$gen_{AE}([x:=e]p) = \{e\} \quad \text{if } x \notin \text{vars}(e),$$

$$gen_{AE}([x:=e]p) = \emptyset \quad \text{if } x \in \text{vars}(e);$$

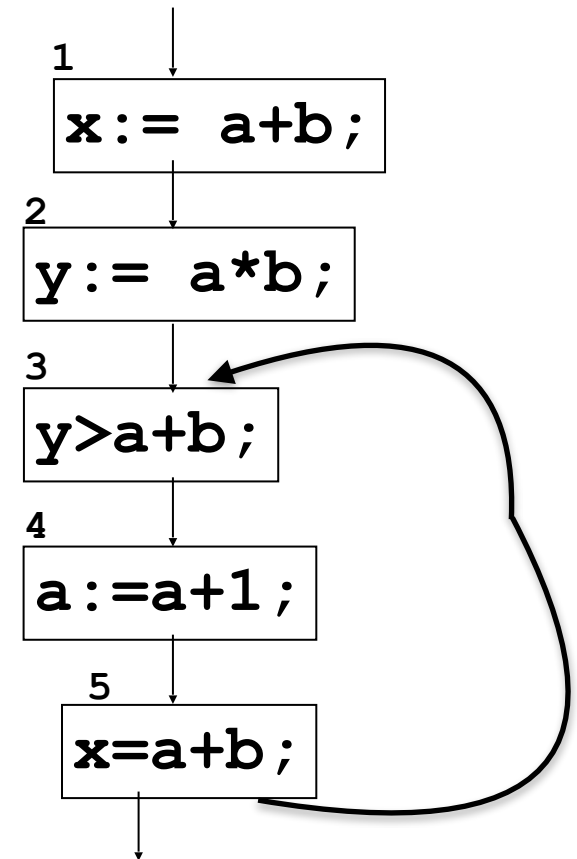
$$gen_{AE}([e1 \triangleright e2]p) = \text{expr}(\{e1, e2\}) \quad \text{where } \text{expr}(S) \text{ returns} \\ \text{the subset of } S \text{ that are expressions}$$

# Example

$x := a + b; y := a * b; \text{ while } y > a + b \text{ do } (a := a + 1; x := a + b)$

$E = \{a + b, a * b, a + 1\}$

| n | $\text{kill}_{AE}(n)$     | $\text{gen}_{AE}(n)$ |
|---|---------------------------|----------------------|
| 1 | $\emptyset$               | $\{a + b\}$          |
| 2 | $\emptyset$               | $\{a * b\}$          |
| 3 | $\emptyset$               | $\{a + b\}$          |
| 4 | $\{a + b, a * b, a + 1\}$ | $\emptyset$          |
| 5 | $\emptyset$               | $\{a + b\}$          |



## Specification

---

- Available expressions analysis is specified by the following equations, for any program point  $p$ :

$$AE_{\text{entry}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is initial} \\ \cap \{AE_{\text{exit}}(q) \mid q \in \text{pre}[p]\} & \text{otherwise} \end{cases}$$

$$AE_{\text{exit}}(p) = (AE_{\text{entry}}(p) \setminus \text{kill}_{AE}(p)) \cup \text{gen}_{AE}(p)$$

## Point a and b to apply Kleene Theorem

To find a solution to the previous equation system we need to apply Kleene Theorem

b)  $(\mathcal{P}(E) \times \mathcal{P}(E))^N, \subseteq^{2N}$  is a finite domain therefore is a CPO, moreover, it has a bottom element

a) The map  $(\mathcal{P}(E) \times \mathcal{P}(E))^N \rightarrow (\mathcal{P}(E) \times \mathcal{P}(E))^N$  defined by  
(assuming 1 is the only initial node)

$$AE(\langle AE_{\text{entry}1}, AE_{\text{exit}1}, \dots, AE_{\text{entry}N}, AE_{\text{exit}N} \rangle) =$$

$$\langle \emptyset, (AE_{\text{entry}1} \setminus \text{kill}_{AE}(1)) \cup \text{gen}_{AE}(1),$$

$$\cap \{AE_{\text{exit}q} \mid q \text{ in pre}[2]\}, (AE_{\text{entry}2} \setminus \text{kill}_{AE}(2)) \cup \text{gen}_{AE}(2),$$

.....

$$\cap \{AE_{\text{exit}q} \mid q \text{ in pre}[N]\}, (AE_{\text{entry}N} \setminus \text{kill}_{AE}(N)) \cup \text{gen}_{AE}(N) \rangle$$

## Point a

a) The map

$$\begin{aligned}
 & AE(\langle AE_{\text{entry}1}, AE_{\text{exit}1}, \dots, AE_{\text{entry}N}, AE_{\text{exit}N} \rangle) = \\
 & \langle \emptyset, (AE_{\text{entry}1} \setminus \text{kill}_{AE}(1)) \cup \text{gen}_{AE}(1), \\
 & \cap \{AE_{\text{exit}q} \mid q \text{ in pre}[2]\}, (AE_{\text{entry}2} \setminus \text{kill}_{AE}(2)) \cup \text{gen}_{AE}(2), \\
 & \dots \\
 & \cap \{AE_{\text{exit}q} \mid q \text{ in pre}[N]\}, (AE_{\text{entry}N} \setminus \text{kill}_{AE}(N)) \cup \text{gen}_{AE}(N) \rangle
 \end{aligned}$$

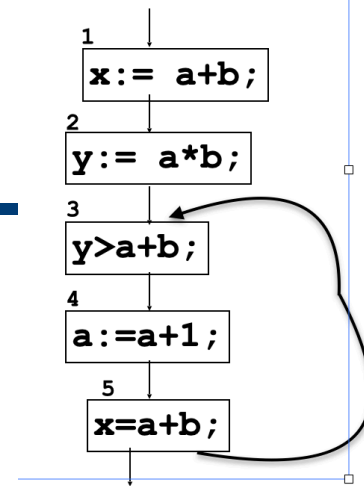
is monotone on the finite domain

$$(\mathcal{P}_{(E)} \times \mathcal{P}_{(E)})^N, \subseteq^{2N}$$

• Example

$$\begin{aligned}
 & AE(\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \dots \rangle) = \langle \emptyset, \{a+b\}, \{\}, \{a*b\}, \{\}, \{a+b\}, \dots \rangle \\
 & \subseteq^{2N} \qquad \qquad \qquad \subseteq^{2N}
 \end{aligned}$$

$$AE(\langle \emptyset, \{a+b\}, \{\}, \{a*b\}, \{\}, \{a+b\}, \dots \rangle) = \langle \emptyset, \{a+b\}, \{a+b\}, \{a*b, a+b\}, \{\}, \{a+b\}, \dots \rangle$$



## Which fix point?

---

AE is a definite analysis:

if  $e \in \text{AE}_{\text{entry}}(p)$  then  $e$  is really available in entry to  $p$

the converse does not hold

- Any fixpoint of the above equation system is an under-approximation of really available expressions.

Between all fix points, we are thus interested in computing the **greatest fixpoint** (the more precise approximation)

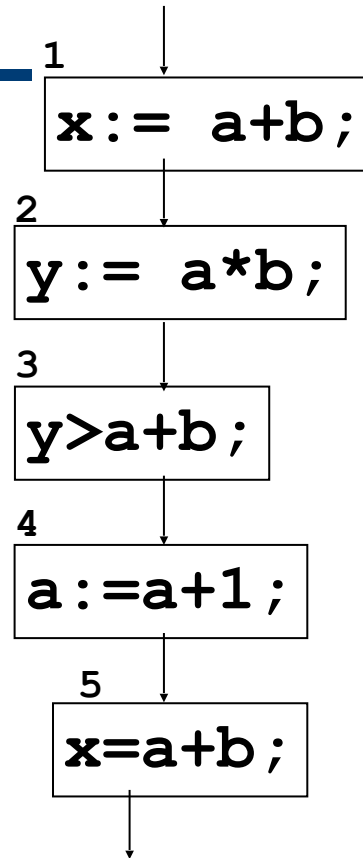
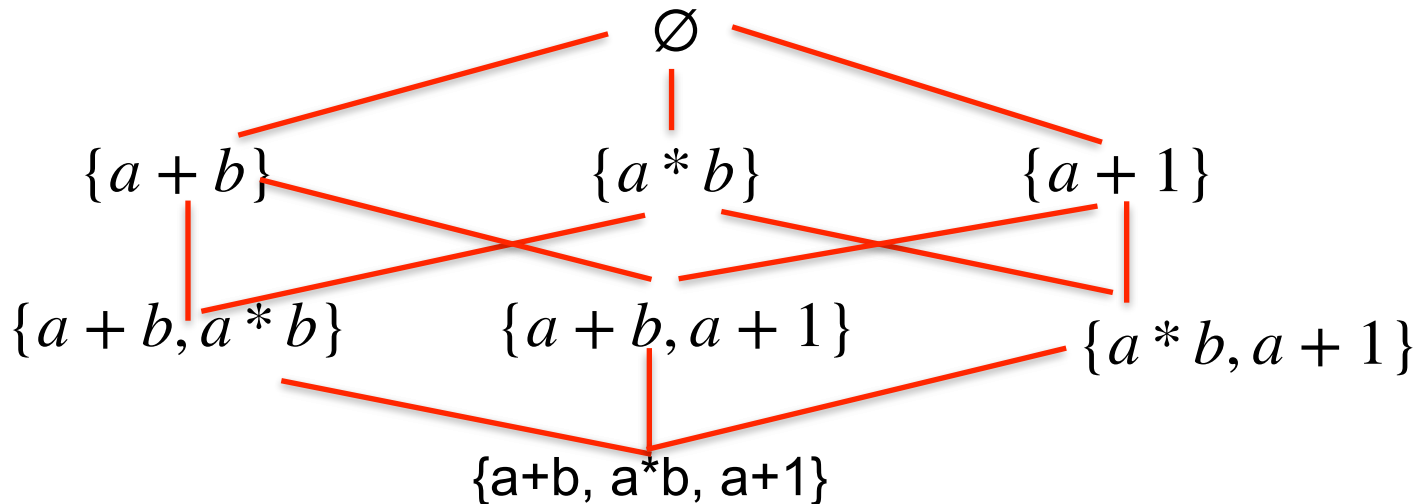
Also, observe that this is a **forward** analysis.

# How to compute a greatest fix point?

Using Kleene's theorem with reverse order.

$$(\mathcal{P}(\mathbf{E}) \times \mathcal{P}(\mathbf{E}))^{\mathbb{N}}, \supseteq^{2N},$$

Example



# Computing the greatest fix point

The starting point, for all n  
 $AE_{\text{entry}}(n) = AE_{\text{exit}}(n) = \{a+b, a*b, a+1\}$

$x := a+b; y := a*b; \text{ while } y > a+b \text{ do } (a := a+1; x := a+b)$

$E = \{a+b, a*b, a+1\}$

| n | kill <sub>AE</sub> (n) | gen <sub>AE</sub> (n) |
|---|------------------------|-----------------------|
| 1 | $\emptyset$            | $\{a+b\}$             |
| 2 | $\emptyset$            | $\{a*b\}$             |
| 3 | $\emptyset$            | $\{a+b\}$             |
| 4 | $\{a+b, a*b, a+1\}$    | $\emptyset$           |
| 5 | $\emptyset$            | $\{a+b\}$             |

$$AE_{\text{entry}}(1) = \emptyset$$

$$AE_{\text{exit}}(1) = \{a+b\}$$

$$AE_{\text{entry}}(2) = \{a+b\}$$

$$AE_{\text{exit}}(2) = \{a+b, a*b\}$$

$$AE_{\text{entry}}(3) = \{a+b, a*b\}$$

$$AE_{\text{exit}}(3) = \{a+b, a*b\}$$

$$AE_{\text{entry}}(4) = \{a+b, a*b\}$$

$$AE_{\text{exit}}(4) = \{\}$$

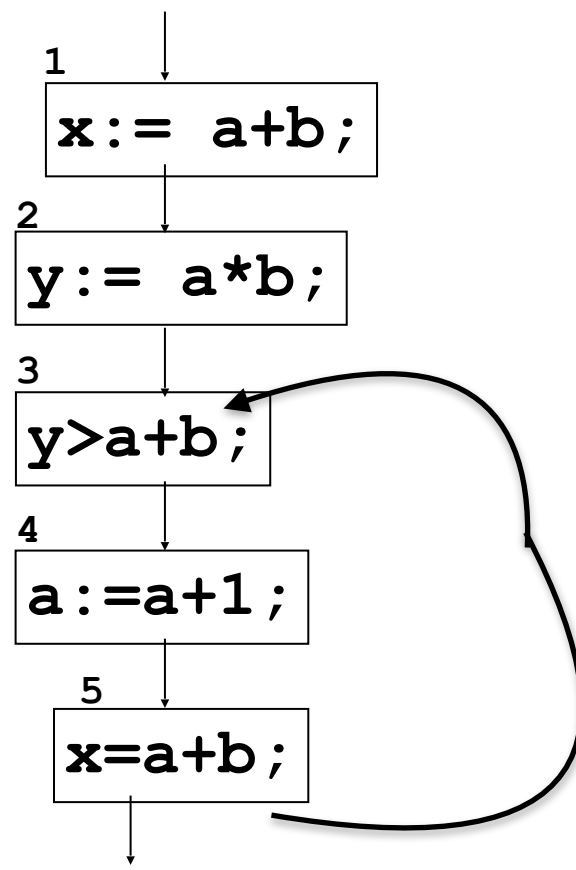
$$AE_{\text{entry}}(5) = \{\}$$

$$AE_{\text{exit}}(5) = \{a+b\}$$

$$AE_{\text{entry}}(p) = \emptyset \text{ if } p \text{ is initial}$$

$$AE_{\text{entry}}(p) = \bigcap \{AE_{\text{exit}}(q) \mid q \text{ in pre}[p]\}$$

$$AE_{\text{exit}}(p) = (AE_{\text{entry}}(p) \setminus \text{kill}_{AE}(p)) \cup \text{gen}_{AE}(p)$$





## Second iteration

$AE_{\text{entry}}(p) = \emptyset$  if  $p$  is initial

$AE_{\text{entry}}(p) = \bigcap \{AE_{\text{exit}}(q) \mid q \text{ in pre}[p]\}$

$AE_{\text{exit}}(p) = (AE_{\text{entry}}(p) \setminus \text{kill}_{AE}(p)) \cup \text{gen}_{AE}(p)$

### Previous iteration

| n | $AE_{\text{entry}}(n)$ | $AE_{\text{exit}}(n)$ |
|---|------------------------|-----------------------|
| 1 | $\emptyset$            | $\{a+b\}$             |
| 2 | $\{a+b\}$              | $\{a+b, a*b\}$        |
| 3 | $\{a+b, a*b\}$         | $\{a+b, a*b\}$        |
| 4 | $\{a+b, a*b\}$         | $\emptyset$           |
| 5 | $\emptyset$            | $\{a+b\}$             |

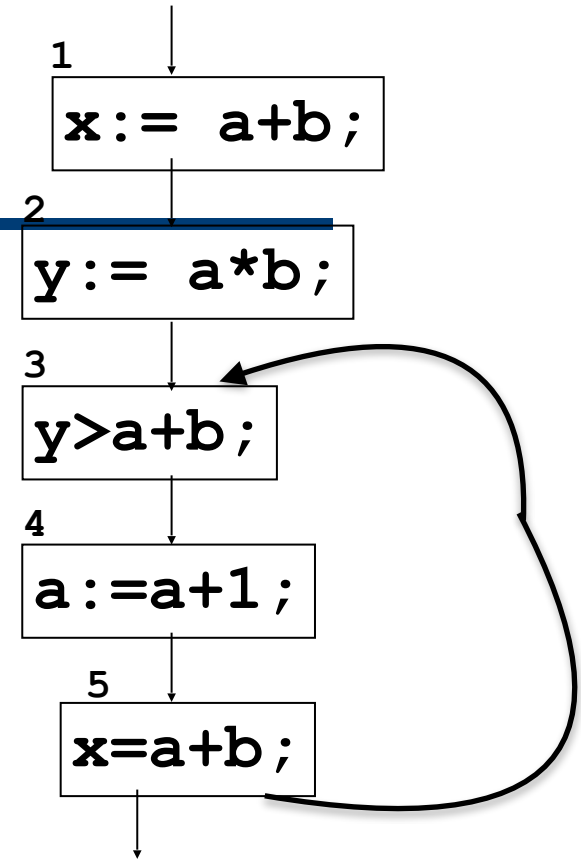
$AE_{\text{exit}}(1) = AE_{\text{entry}}(1) \cup \{a+b\}$

$AE_{\text{exit}}(2) = AE_{\text{entry}}(2) \cup \{a*b\}$

$AE_{\text{exit}}(3) = AE_{\text{entry}}(3) \cup \{a+b\}$

$AE_{\text{exit}}(4) = AE_{\text{entry}}(4) - \{a+b, a*b, a+1\}$

$AE_{\text{exit}}(5) = AE_{\text{entry}}(5) \cup \{a+b\}$



| n | $AE_{\text{entry}}(n)$ | $AE_{\text{exit}}(n)$ |
|---|------------------------|-----------------------|
| 1 | $\emptyset$            | $\{a+b\}$             |
| 2 | $\{a+b\}$              | $\{a+b, a*b\}$        |
| 3 | $\{a+b\}$              | $\{a+b\}$             |
| 4 | $\{a+b\}$              | $\emptyset$           |
| 5 | $\emptyset$            | $\{a+b\}$             |

# Third iteration and Greatest Fixpoint

$AE_{entry}(p) = \emptyset$  if  $p$  is initial

$AE_{entry}(p) = \bigcap \{AE_{exit}(q) \mid q \text{ in } pre[p]\}$

$AE_{exit}(p) = (AE_{entry}(p) \setminus kill_{AE}(p)) \cup gen_{AE}(p)$

Previous iteration

| n | $AE_{entry}(n)$ | $AE_{exit}(n)$ |
|---|-----------------|----------------|
| 1 | $\emptyset$     | $\{a+b\}$      |
| 2 | $\{a+b\}$       | $\{a+b, a*b\}$ |
| 3 | $\{a+b\}$       | $\{a+b\}$      |
| 4 | $\{a+b\}$       | $\emptyset$    |
| 5 | $\emptyset$     | $\{a+b\}$      |

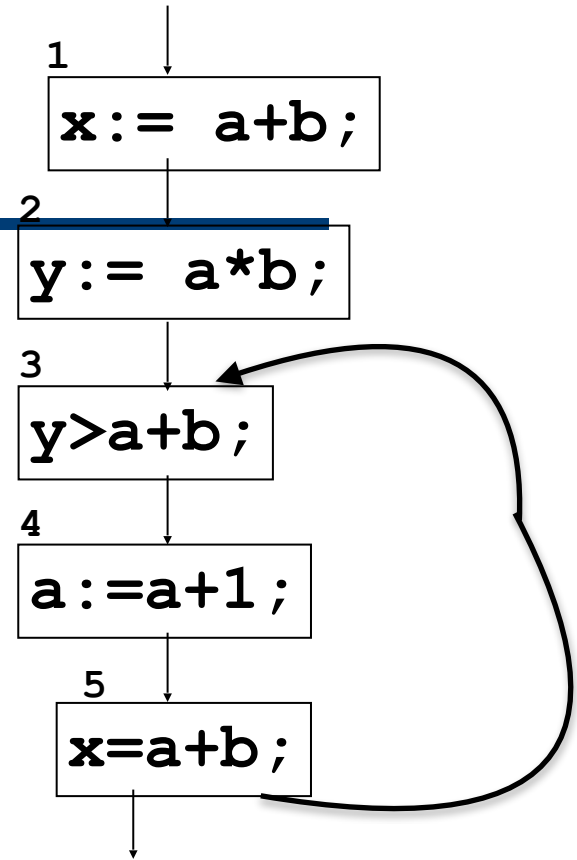
$AE_{exit}(1) = AE_{entry}(1) \cup \{a+b\}$

$AE_{exit}(2) = AE_{entry}(2) \cup \{a*b\}$

$AE_{exit}(3) = AE_{entry}(3) \cup \{a+b\}$

$AE_{exit}(4) = AE_{entry}(4) - \{a+b, a*b, a+1\}$

$AE_{exit}(5) = AE_{entry}(5) \cup \{a+b\}$

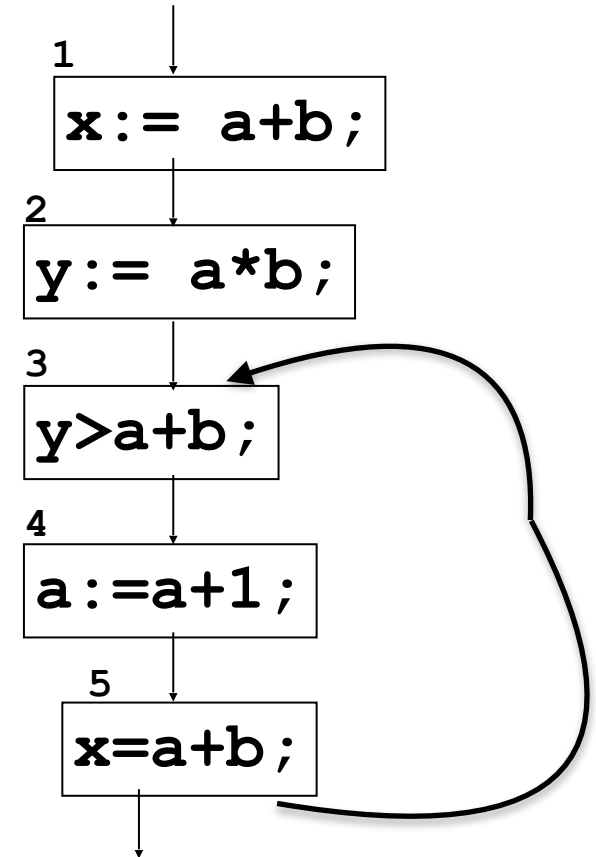


| n | $AE_{entry}(n)$ | $AE_{exit}(n)$ |
|---|-----------------|----------------|
| 1 | $\emptyset$     | $\{a+b\}$      |
| 2 | $\{a+b\}$       | $\{a+b, a*b\}$ |
| 3 | $\{a+b\}$       | $\{a+b\}$      |
| 4 | $\{a+b\}$       | $\emptyset$    |
| 5 | $\emptyset$     | $\{a+b\}$      |

# Result

$x := a + b; y := a * b; \text{ while } y > a + b \text{ do } (a := a + 1; x := a + b)$

| n | $AE_{\text{entry}}(n)$ | $AE_{\text{exit}}(n)$ |
|---|------------------------|-----------------------|
| 1 | $\emptyset$            | $\{a+b\}$             |
| 2 | $\{a+b\}$              | $\{a+b, a*b\}$        |
| 3 | $\{a+b\}$              | $\{a+b\}$             |
| 4 | $\{a+b\}$              | $\emptyset$           |
| 5 | $\emptyset$            | $\{a+b\}$             |



# Very Busy Expression Analysis

- An expression  $exp$  is **very busy** at the exit of a program point  $p$  if in **any execution path** outgoing from  $p$  the expression  $exp$  is used before that any variable that occurs in  $exp$  is (re-)defined.
- For instance, in this program

if  $[a > b]^1$  then  $([x := b - a]^2; [y := a - b]^3)$  else  $([y := b - a]^4; [x := a - b]^5)$

both expressions  $a - b$  and  $b - a$  are very busy at point 1.

# Usage

- This analysis can be used to evaluate a very busy expression at some program point in order to store its value for future uses thus avoiding re-computations.
- This technique is also called expression “hoisting”.

```
if(expr)
    x=i+j;
else
    y=(i+j)*2;
```



```
temp=i+j;
if(expr)
    x=temp;
else
    y=temp*2;
```

```
for (i=0; i<100; i++)
    a[i]=x+y;
```



```
temp=x+y;
for (i=0; i<100; i++)
    a[i]=temp;
```

This may save time although it does not necessarily save space

# Available vs Very Busy Expressions

- Available Expressions (AE)
  - Already computed on all paths reaching a point
  - Forward, must analysis
- Very Busy Expressions (VBE)
  - Will be computed on all paths from a point
  - Backward, must analysis

# Examples and Applications

- Available but NOT Very Busy

$x = a + b;$

.....

$p \rightarrow a+b$  available but may not be used later

- Very Busy but NOT Available

$p \rightarrow a+b$  very busy but not computed before

.....

$\text{if}(c)\{x=a+b;\} \text{ else } \{y=a+b;\}$

- Applications
  - AE: Common Subexpression Elimination
  - VBE: Moving computation of expressions earlier where they can be reused

# kill and gen

- An expression  $exp$  is VB-killed by some program point  $p$  (notation:  $exp \in kill_{VB}(p)$ ) if some variable occurring in  $exp$  is modified (i.e. defined) in  $p$ . Note that  $kill_{VB} = kill_{AE}$ .
- An expression  $exp$  is VB-generated in some program point  $p$  (notation:  $exp \in gen_{VB}(p)$ ) if  $exp$  is evaluated in  $p$  (note that  $gen_{VB} \neq gen_{AE}$ , for instance in the case  $x := x + 1$ ).
- if  $[a > b]^1$  then  $([x := b - a]^2 ; [y := a - b]^3)$  else  $([y := b - a]^4 ; [x := a - b]^5)$

| $n$ | $kill_{VB}(n)$ | $gen_{VB}(n)$ |
|-----|----------------|---------------|
| 1   | $\emptyset$    | $\emptyset$   |
| 2   | $\emptyset$    | $\{b - a\}$   |
| 3   | $\emptyset$    | $\{a - b\}$   |
| 4   | $\emptyset$    | $\{b - a\}$   |
| 5   | $\emptyset$    | $\{a - b\}$   |



# Specification

- For any program point  $p$ :

$$VB_{\text{exit}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is final} \\ \cap \{VB_{\text{entry}}(q) \mid q \in \text{post}[p]\} & \text{otherwise} \end{cases}$$

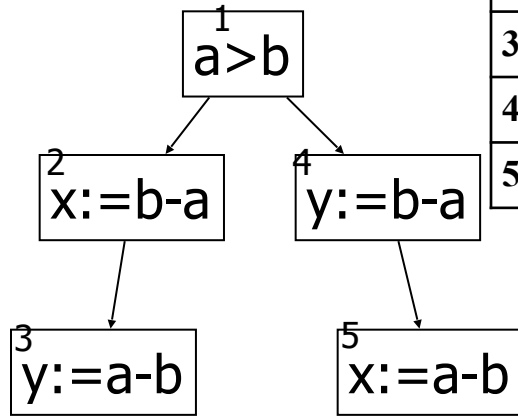
$$VB_{\text{entry}}(p) = (VB_{\text{exit}}(p) \setminus \text{kill}_{VB}(p)) \cup \text{gen}_{VB}(p)$$

# Very Busy Expression Analysis

- VB analysis is definite: if  $\text{exp} \in \text{VB}_{\text{exit}}(p)$  then exp is really very busy in exit from p (while the vice versa does not hold)
- VB analysis is backward. Notice that no expression is very busy in exit from a terminal node of the CFG.

# Very Busy Expressions

| n | kill <sub>VB</sub> (n) | gen <sub>VB</sub> (n) |
|---|------------------------|-----------------------|
| 1 | ∅                      | ∅                     |
| 2 | ∅                      | {b-a}                 |
| 3 | ∅                      | {a-b}                 |
| 4 | ∅                      | {b-a}                 |
| 5 | ∅                      | {a-b}                 |



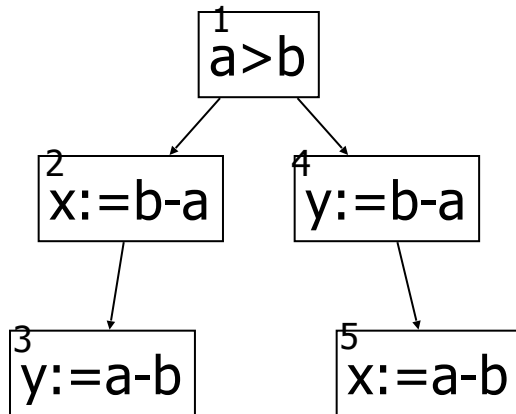
$$\text{VB}_{\text{exit}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is final} \\ \cap \{ \text{VB}_{\text{entry}}(q) \mid q \in \text{post}[p] \} & \text{otherwise} \end{cases}$$

$$\text{VB}_{\text{entry}}(p) = (\text{VB}_{\text{exit}}(p) \setminus \text{kill}_{\text{VB}}(p)) \cup \text{gen}_{\text{VB}}(p)$$

| Node      | 1          | 2          | 3          | 4          | 5          |
|-----------|------------|------------|------------|------------|------------|
| Entry (0) | {a-b, b-a} | {a-b, b-a} | {a-b, b-a} | {a-b, b-a} | {a-b, b-a} |
| Exit (0)  | {a-b, b-a} | {a-b, b-a} | ∅          | {a-b, b-a} | ∅          |
| Entry (1) | {a-b, b-a} | {a-b, b-a} | {a-b}      | {a-b, b-a} | {a-b}      |
| Exit (1)  | {a-b, b-a} | {a-b, b-a} | ∅          | {a-b, b-a} | ∅          |
| Entry (2) | {a-b, b-a} | {a-b, b-a} | {a-b}      | {a-b, b-a} | {a-b}      |
| Exit (2)  | {a-b, b-a} | {a-b}      | ∅          | {a-b}      | ∅          |

# Greatest Fixpoint

if  $[a > b]^1$  then  $([x := b - a]^2 ; [y := a - b]^3)$  else  $([y := b - a]^4 ; [x := a - b]^5)$



| n        | $VB_{\text{entry}}(n)$ | $VB_{\text{exit}}(n)$ |
|----------|------------------------|-----------------------|
| <b>1</b> | $\{a-b, b-a\}$         | $\{a-b, b-a\}$        |
| <b>2</b> | $\{a-b, b-a\}$         | $\{a-b\}$             |
| <b>3</b> | $\{a-b\}$              | $\emptyset$           |
| <b>4</b> | $\{a-b, b-a\}$         | $\{a-b\}$             |
| <b>5</b> | $\{a-b\}$              | $\emptyset$           |

# A Dataflow Analysis Framework

- The above dataflow analyses (Reaching Definitions, Available Expressions, Very Busy Expressions, Live Variables) reveal many similarities.
- One major advantage of a unifying framework of dataflow analysis lies in the design of a generic analysis algorithm that can be instantiated in order to compute different dataflow analyses.

# Catalogue of Dataflow Analyses

---

Dataflow analyses differ along two orthogonal dimensions: direction and approximation (may vs must).

|                                                                                                             | <i>Possible Analysis</i><br><b>Semantics <math>\subseteq</math> Analysis</b><br><br>May analysis: property holds on <i>some</i> execution path | <i>Definite Analysis</i><br><b>Analysis <math>\subseteq</math> Semantics</b><br><br>Must analysis: property holds on <i>all</i> execution paths |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Forward</i><br><b>Information flows along control flow (predecessors <math>\rightarrow</math> node)</b>  | Reaching definitions                                                                                                                           | Available expressions                                                                                                                           |
| <i>Backward</i><br><b>Information flows against control flow (successors <math>\rightarrow</math> node)</b> | Live variables                                                                                                                                 | Very busy expressions                                                                                                                           |

# Possible/Forward: Reaching Definitions

**Goal:** compute which definitions may reach a program point

$$I = \{(x, ?) \mid x \in \mathbf{Vars}\}$$

$$RD_{\text{entry}}(p) = \begin{cases} & \text{if } p \text{ is initial} \\ \cup \{RD_{\text{exit}}(q) \mid q \in \text{pre}[p]\} & \text{otherwise} \end{cases}$$

$$RD_{\text{exit}}(p) = (RD_{\text{entry}}(p) - \text{kill}_{RD}(p)) \cup \text{gen}_{RD}(p)$$

Forward + May analysis (uses union over predecessors)

# Definite/Forward: Available Expressions

**Goal:** compute expressions that are definitely available (already computed on all paths)

$$AE_{\text{entry}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is initial} \\ \bigcap \{AE_{\text{exit}}(q) \mid q \in \text{pre}[p]\} & \text{otherwise} \end{cases}$$

$$AE_{\text{exit}}(p) = (AE_{\text{entry}}(p) - \text{kill}_{AE}(p)) \cup \text{gen}_{AE}(p)$$

Forward + Must analysis (uses intersection over predecessors)



# Definite/Backward: Very Busy Expressions

**Goal:** compute expressions that will definitely be used in the future before being redefined

$$VB_{\text{exit}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is final} \\ \bigcap \{VB_{\text{entry}}(q) \mid q \in \text{post}[p]\} & \text{otherwise} \end{cases}$$

$$VB_{\text{entry}}(p) = (VB_{\text{exit}}(p) - \text{kill}_{VB}(p)) \cup \text{gen}_{VB}(p)$$

Backward + Must analysis (uses intersection over successors)

# Possible/Backward: Live Variables Analysis

**Goal:** compute variables that may be used in the future

$$LV_{\text{exit}}(p) = \begin{cases} \emptyset & \text{if } p \text{ is final} \\ U\{LV_{\text{entry}}(q) \mid q \in \text{post}[p]\} & \text{otherwise} \end{cases}$$

$$LV_{\text{entry}}(p) = (LV_{\text{exit}}(p) - \text{kill}_{LV}(p)) \cup \text{gen}_{LV}(p)$$

Backward + May analysis (uses union over successors)

# A common pattern

All dataflow analyses can be expressed as instances of the same equation schema

$$A[n] = \bigoplus_{m \in N(n)} B[m] \qquad B[n] = f_n(A[n])$$

## Interpretation:

- $A[n]$ : information before the transfer function (IN/OUT)
- $B[n]$ : information after the transfer function (OUT/IN)
- $N(n)$ : nodes from which information flows (SUCC/PRED)
- $\oplus$ : merge operator ( $\cup$  for may,  $\cap$  for must)

# Forward vs Backward

Direction determines where information is collected from

$$A[n] = \bigoplus_{m \in N(n)} B[m] \qquad B[n] = f_n(A[n])$$

Forward:  $A = IN$ ,  $B = OUT$ ,  $N(n) = Pred(n)$

Backward:  $A = OUT$ ,  $B = IN$ ,  $N(n) = Succ(n)$

# Possible vs Definite

Approximation determines how paths are combined

$$A[n] = \bigoplus_{m \in N(n)} B[m] \qquad B[n] = f_n(A[n])$$

- If  $\bigoplus$  is set **intersection** then we aim at computing the **largest** set that satisfies these equations on **any** execution path that ends in (exits from) some node: hence, this is a **definite** (or **must**) analysis
- If  $\bigoplus$  is set **union** then we aim at computing the **smallest** set that satisfies these equations on **some** execution path that ends in (exits from) some node: hence, this is a **possible** (or **may**) analysis

# Unifying view of Dataflow analyses

Every dataflow analysis is defined by four design choices:

**Direction:** forward / backward

**Property:** what information is tracked

**Merge operator:** union (may) / intersection (must)

**Transfer function:** gen/kill rules

All dataflow analyses are instances of a fixpoint computation over a graph.